AI VIETNAM
All-in-One Course
(TA Session)

PySpark for Data Processing

Advanced Class: Data Science

Nguyen-Thuan Duong – TA

Truong-Binh Duong - STA

Year 2024

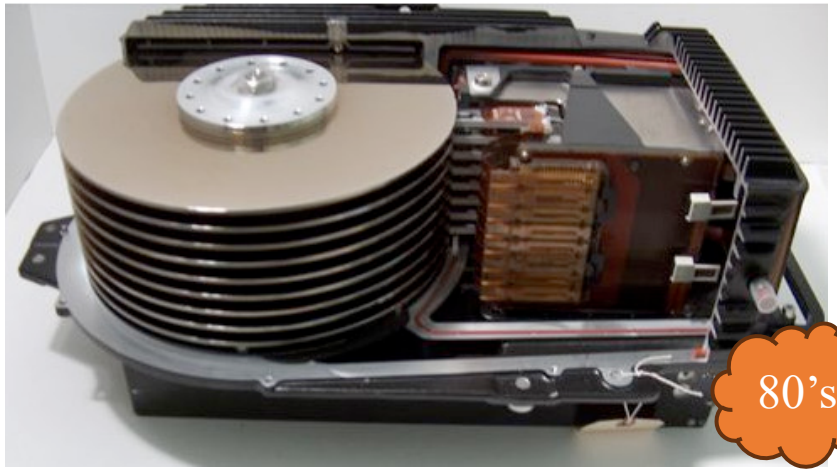
Outline

- Introduction
- Big Data
- Spark
- PySpark
- Hands-on
- Question

Introduction

Introduction

❖ Hard disk



80’s

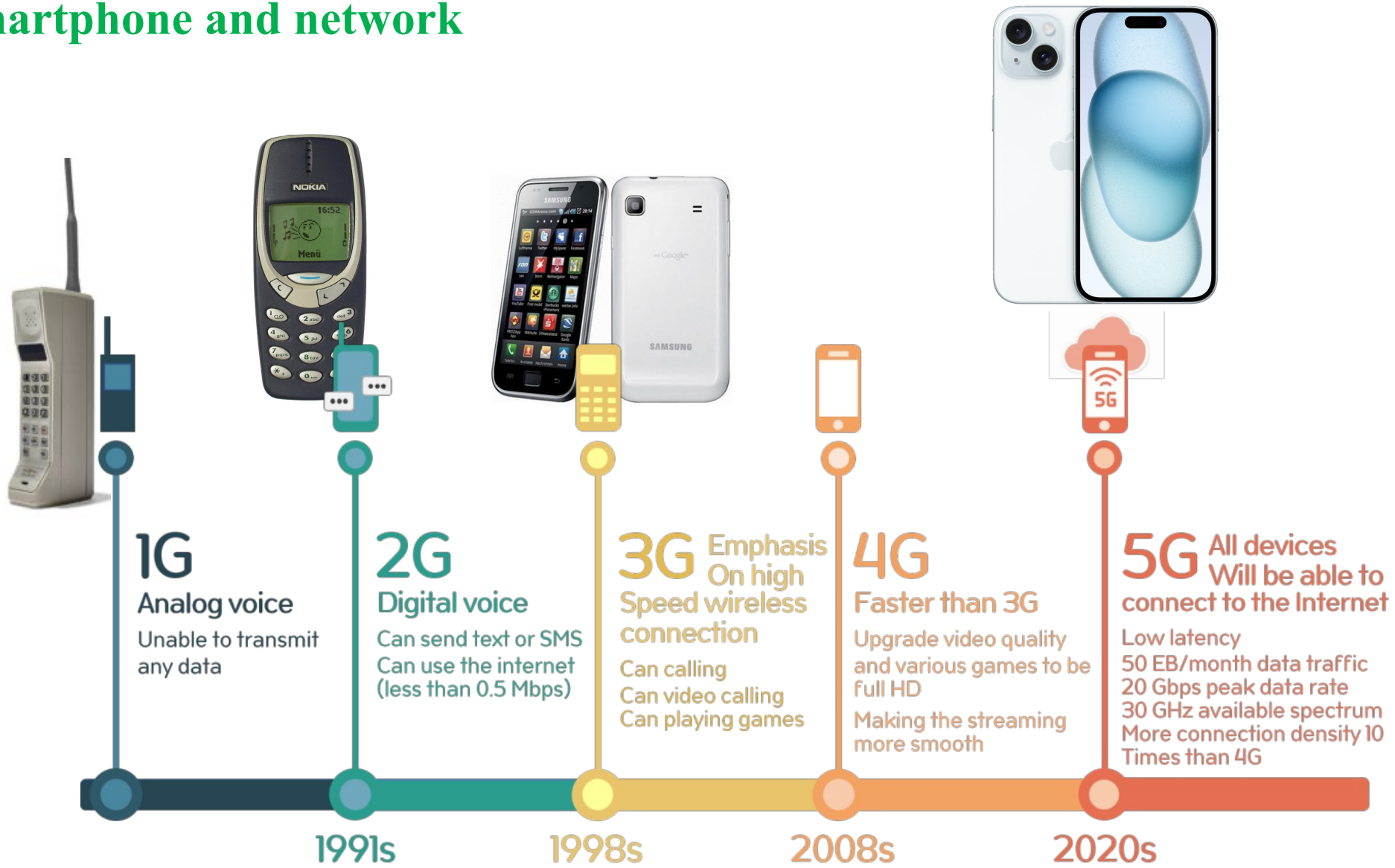
2011

90’s

2000’s

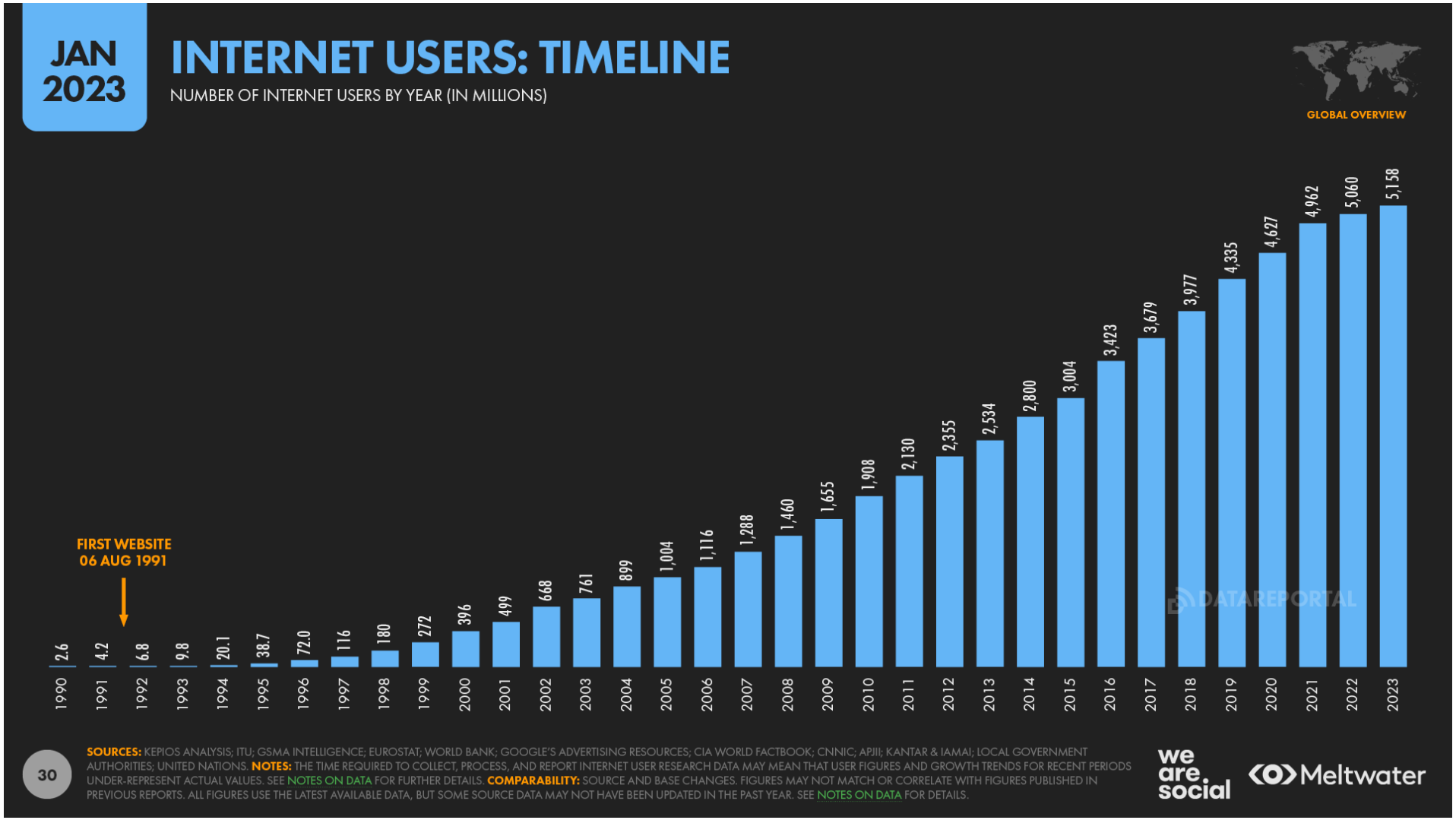
Introduction

❖ Smartphone and network



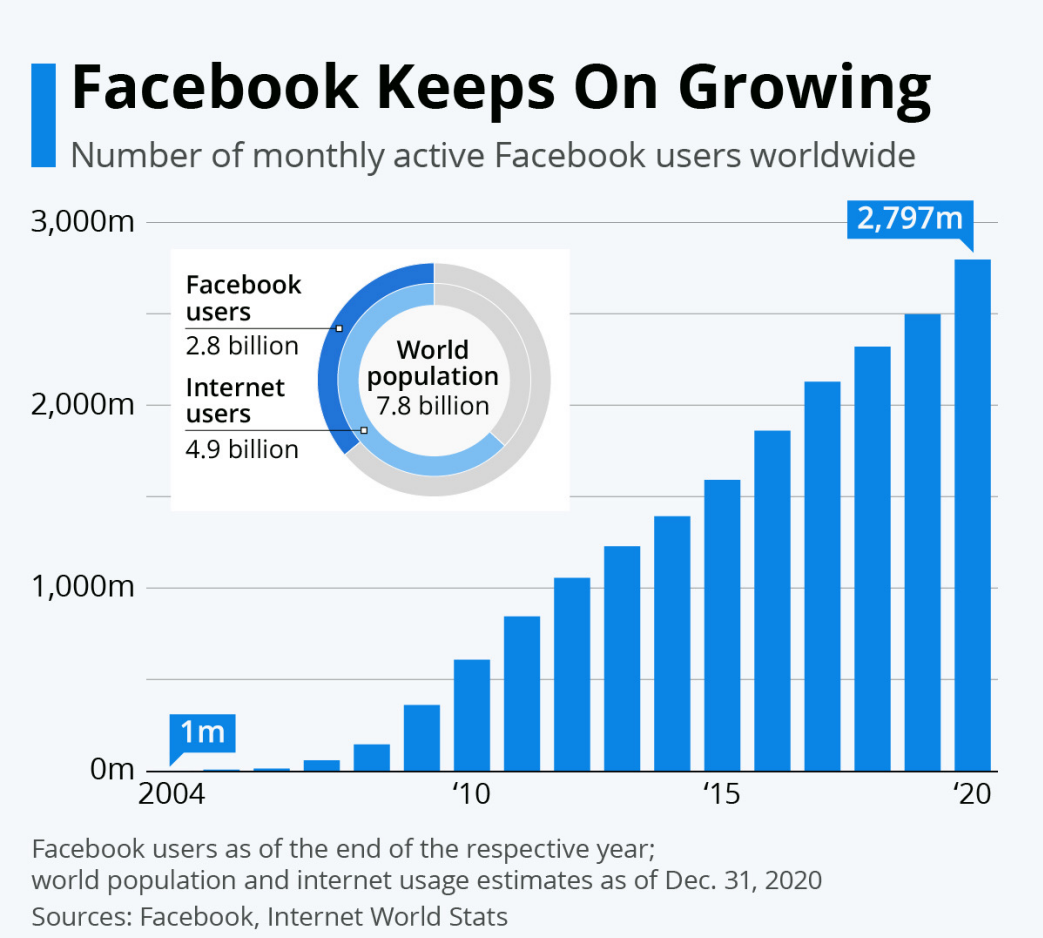
Introduction

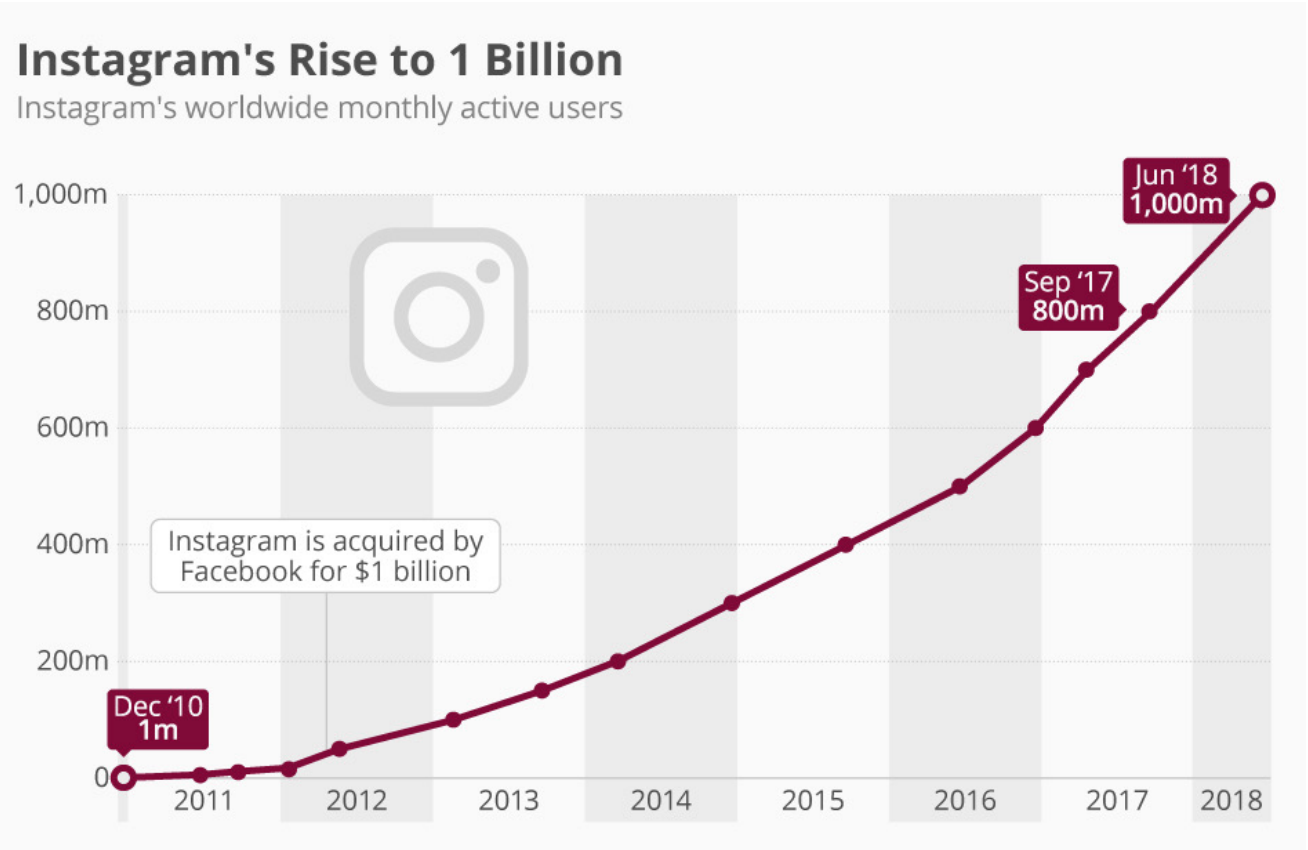
❖ The Internet



Introduction

❖ Social media



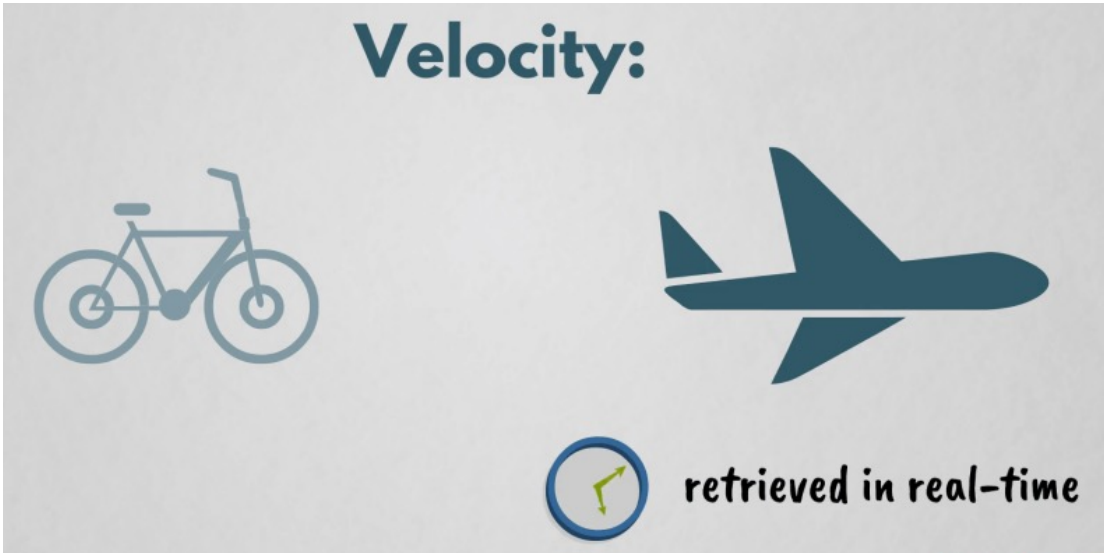


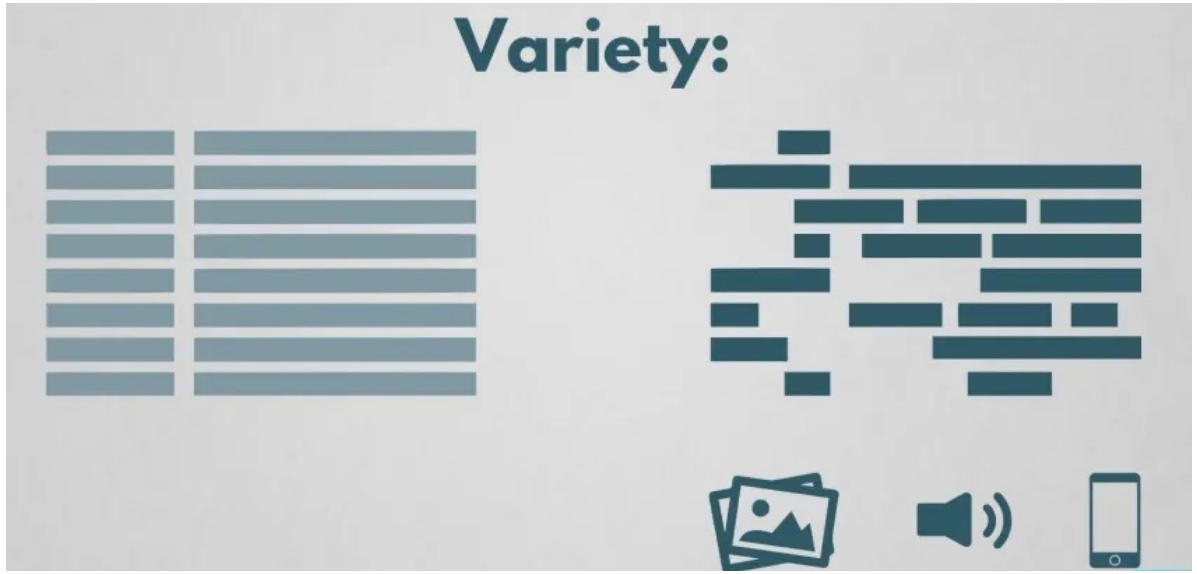
Introduction

❖ Causes

Introduction

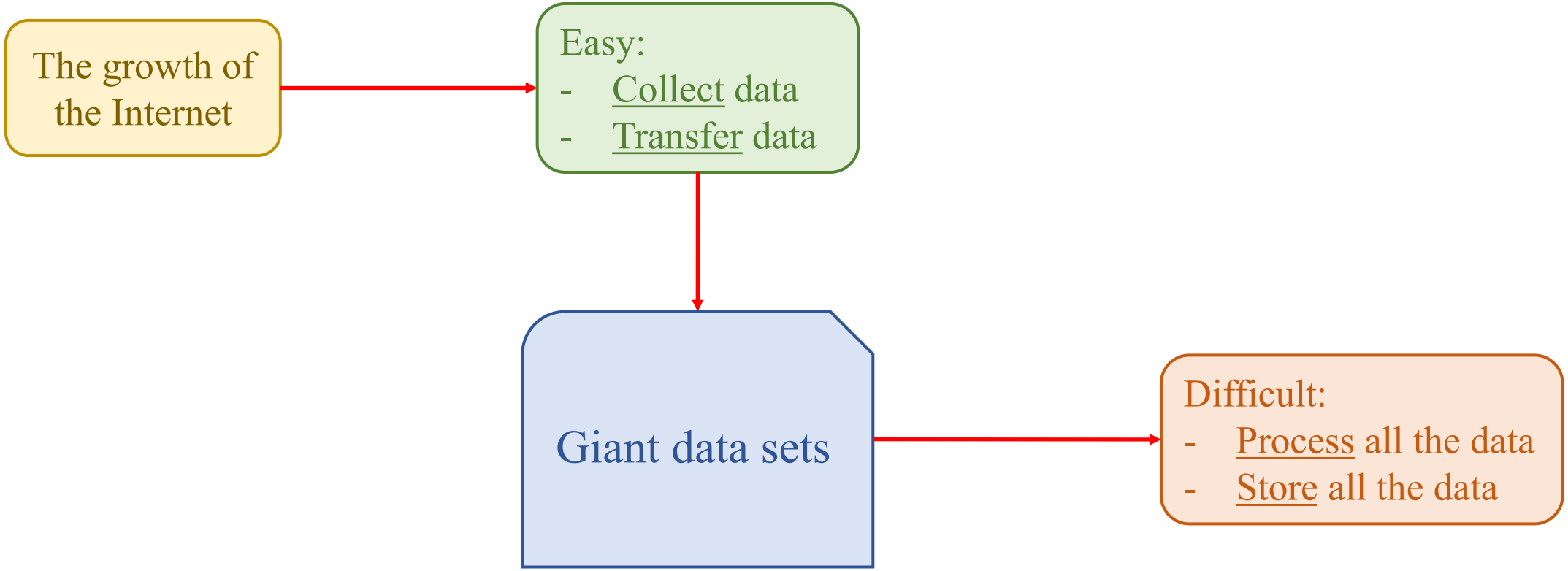
❖ Challenges





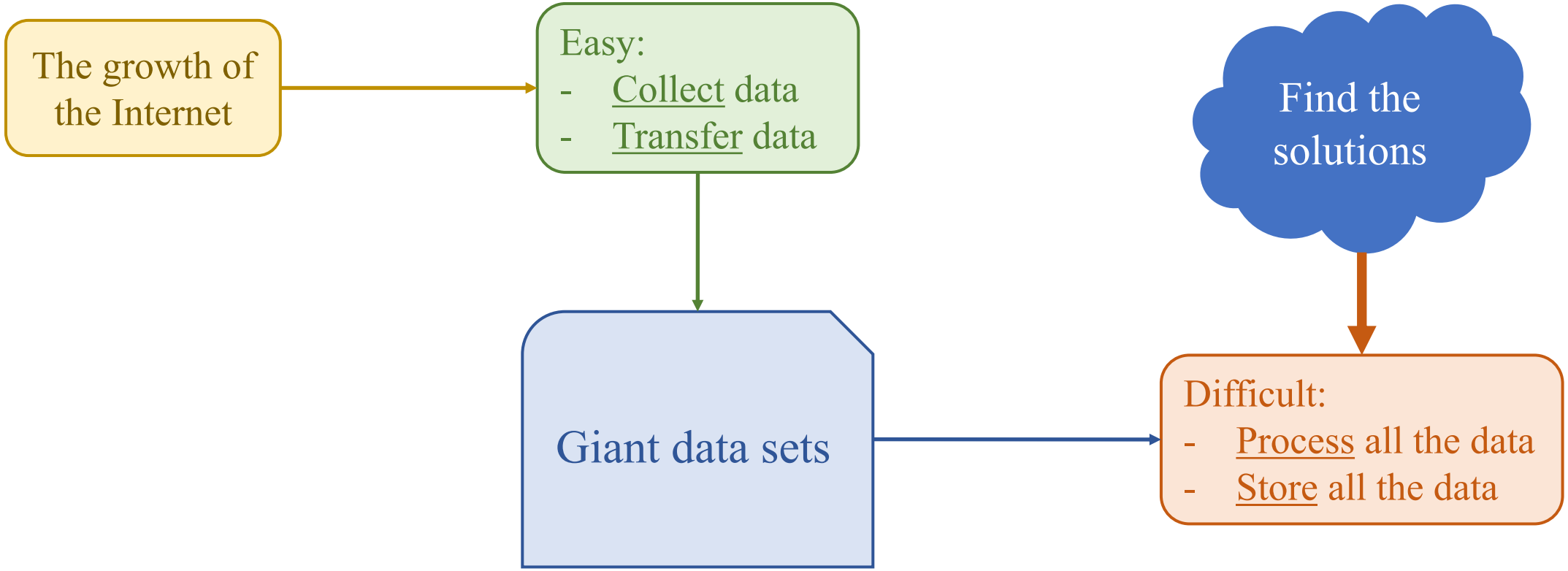
Introduction

❖ Challenges



Introduction

❖ Motivation

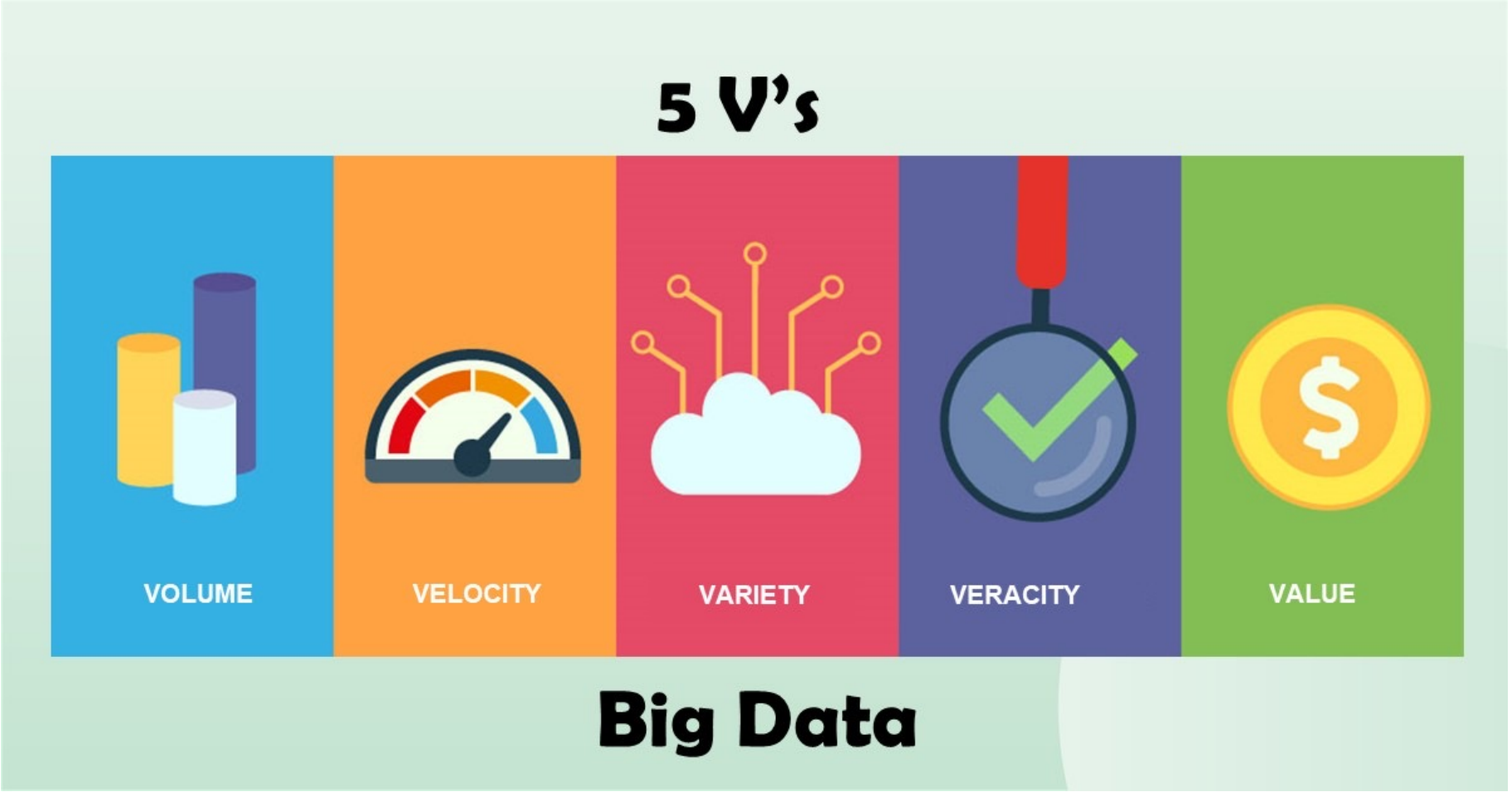


Big Data

Big Data

❖ Getting Started

Big data refers to extremely large and complex datasets that are too large or complex to be processed by traditional data processing applications.



Big Data

❖ Volume

20,000 Servers In 18 Months
It also suggests that Facebook has added about 20,000 servers since early 2008, which explains why it borrowed $100 million in May 2008 to fund server purchases.

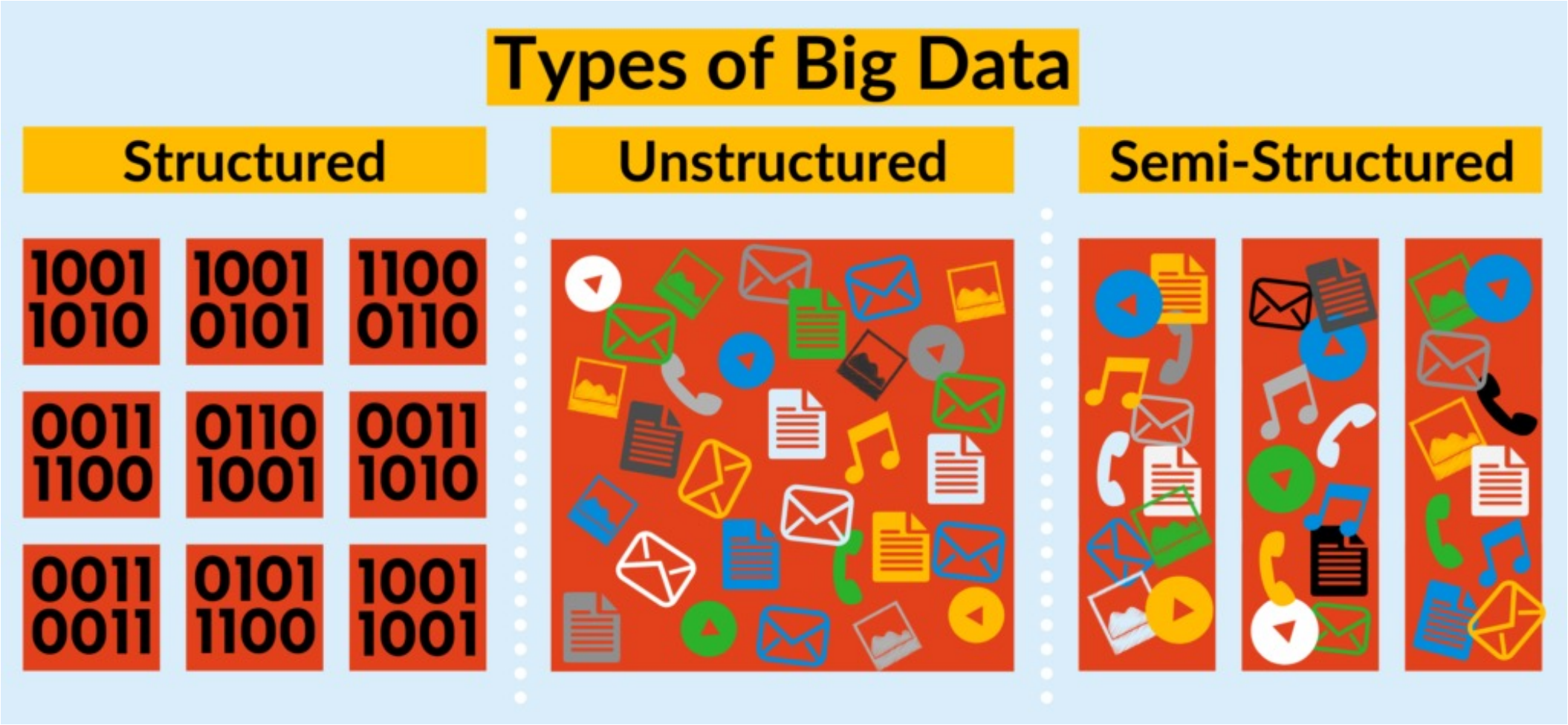
Rothschild also shared some huge numbers associated with Facebook's photo storage operation, which now stores 80 billion images (20 billion images, each in four sizes). Rothschild said the real challenge isn't storage, but delivery. "We serve up 600,000 photos a second," he said.

25 Terabytes of Log Data - Daily
The amount of log data amassed in Facebook's operations is staggering. Rothschild said Facebook manages more than 25 terabytes of data per day in logging data, which he said was the equivalent of about 1,000 times the volume of mail delivered daily by the U.S. Postal Service.

COMPUTER MEMORY UNITS #1		
1 BIT	=	Binary Digit
8 BITS	=	1 Byte
1024 BYTES	=	1 KB (kilo Byte)
1024 KB	=	1 MB (Mega Byte)
1024 MB	=	1 GB (Giga Byte)
1024 GB	=	1 TB (Tera Byte)
1024 TB	=	1 PB (Pera Byte)

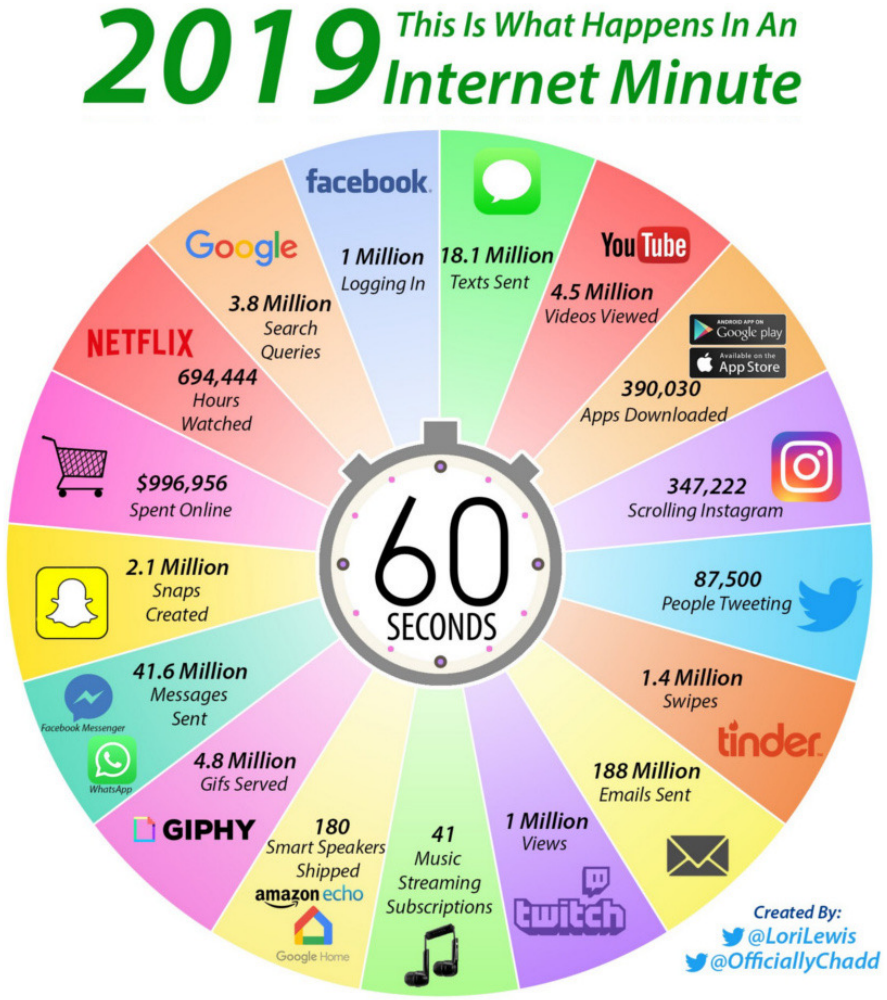
Big Data

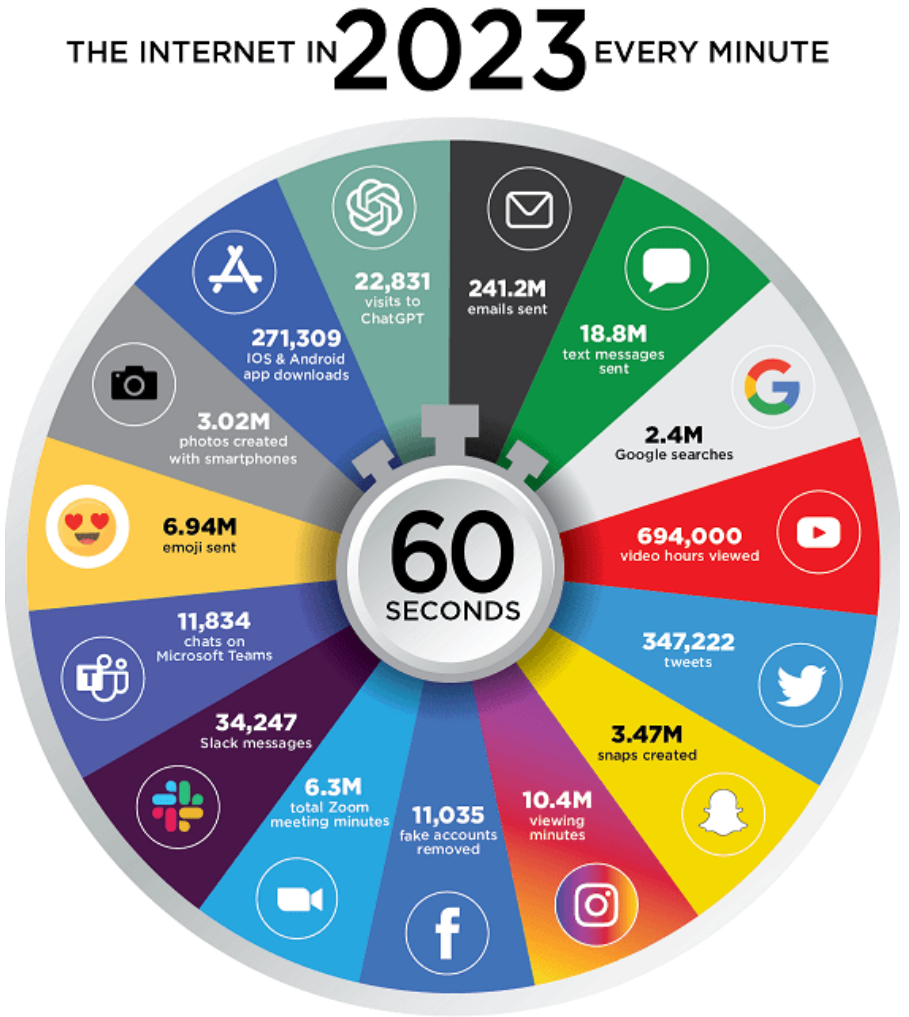
❖ Variaty



Big Data

❖ Velocity





Big Data

❖ Veracity

Data quality and validity are essential to effective Big Data projects.
If the data is not accurate or reliable than the expected benefits of the Big Data initiative will be lost.

Big Data

❖ Veracity

Imagen's training data was drawn from several pre-existing datasets of image and English alt-text pairs. A subset of this data was filtered to removed noise and undesirable content, such as pornographic imagery and toxic language. However, a recent audit of one of our data sources, LAION-400M [61], uncovered a wide range of inappropriate content including pornographic imagery, racist slurs, and harmful social stereotypes [4]. This finding informs our assessment that Imagen is not suitable for public use at this time and also demonstrates the value of rigorous dataset audits and comprehensive dataset documentation (e.g. [23, 45]) in informing consequent decisions about the model's appropriate and safe use. Imagen also relies on text encoders trained on uncurated web-scale data, and thus inherits the social biases and limitations of large language models [5, 3, 50].

Our primary aim with Imagen is to advance research on generative methods, using text-to-image synthesis as a test bed. While end-user applications of generative methods remain largely out of scope, we recognize the potential downstream applications of this research are varied and may impact society in complex ways. On the one hand, generative models have a great potential to complement, extend, and augment human creativity [30]. Text-to-image generation models, in particular, have the potential to extend image-editing capabilities and lead to the development of new tools for creative practitioners. On the other hand, generative methods can be leveraged for malicious purposes, including harassment and misinformation spread [20], and raise many concerns regarding social and cultural exclusion and bias [67, 62, 68]. These considerations inform our decision to not to release code or a public demo. In future work we will explore a framework for responsible externalization that balances the value of external auditing with the risks of unrestricted open-access.

Reference: https://arxiv.org/pdf/2205.11487

Big Data

❖ Value

Improve Customer Experiences

- Recommendation system
- Customer Segmentation

Cutting Costs

Fraud Detection and Prevention
- Anomaly Detection
- Risk Assessment

Pricing Optimization
- Dynamic Pricing
- Price Sensitivity Analysis

Solving Problems

Identifying Patterns and Trends:
- Predicting Future Outcomes
- Uncovering Hidden Relationships

Data-Driven Decision Making - Informed Choices

Insceasing Revenue

Personalized Marketing
- Targeted Campaigns
- Personalized Recommendations

Product Optimization
- Product Development
- Assortment Planning

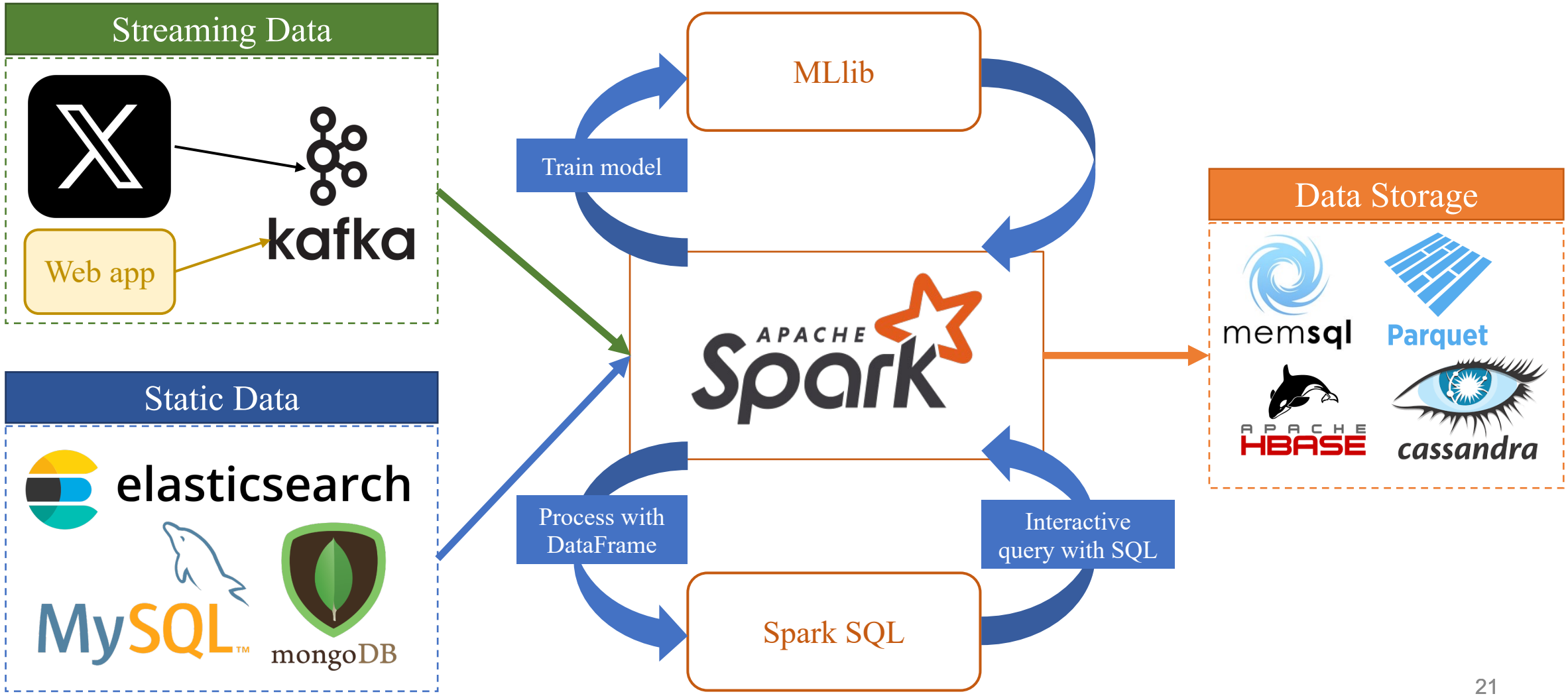
Big Data

❖ Big Data Frameworks

ApacheFlink

Big Data

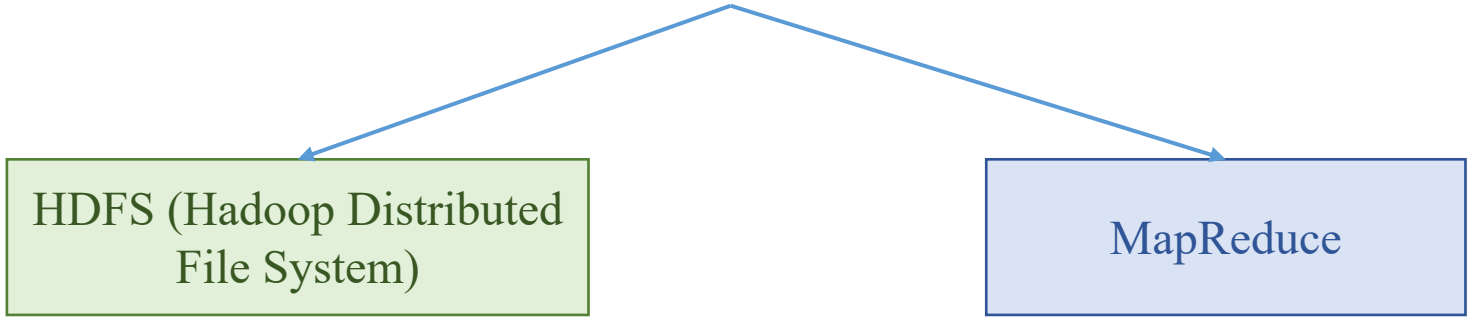
❖Data Pipeline



Big Data

❖ Apache Hadoop

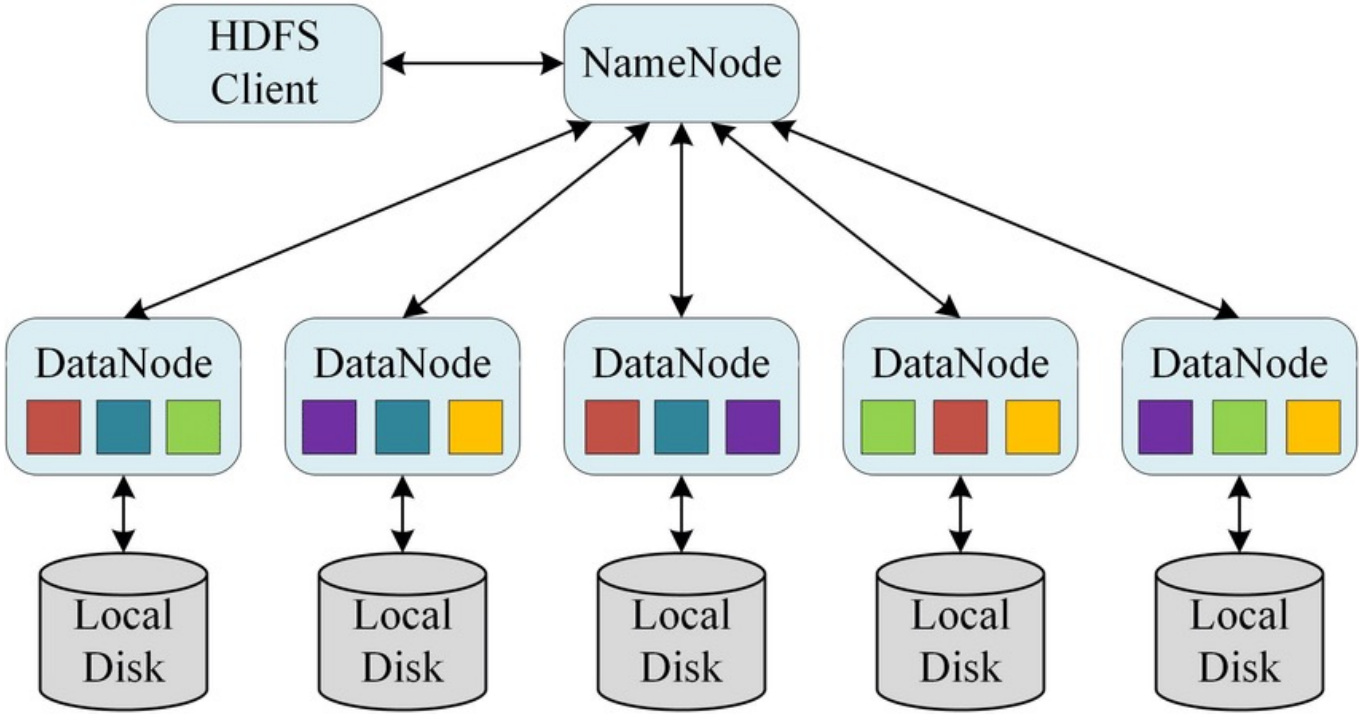
- Hadoop is an open source platform used to store and process large data sets on clusters of computers.
- Components:
  - Hadoop Distributed File System (HDFS)
  - MapReduce



Big Data

❖ HDFS (Hadoop Distributes File System)

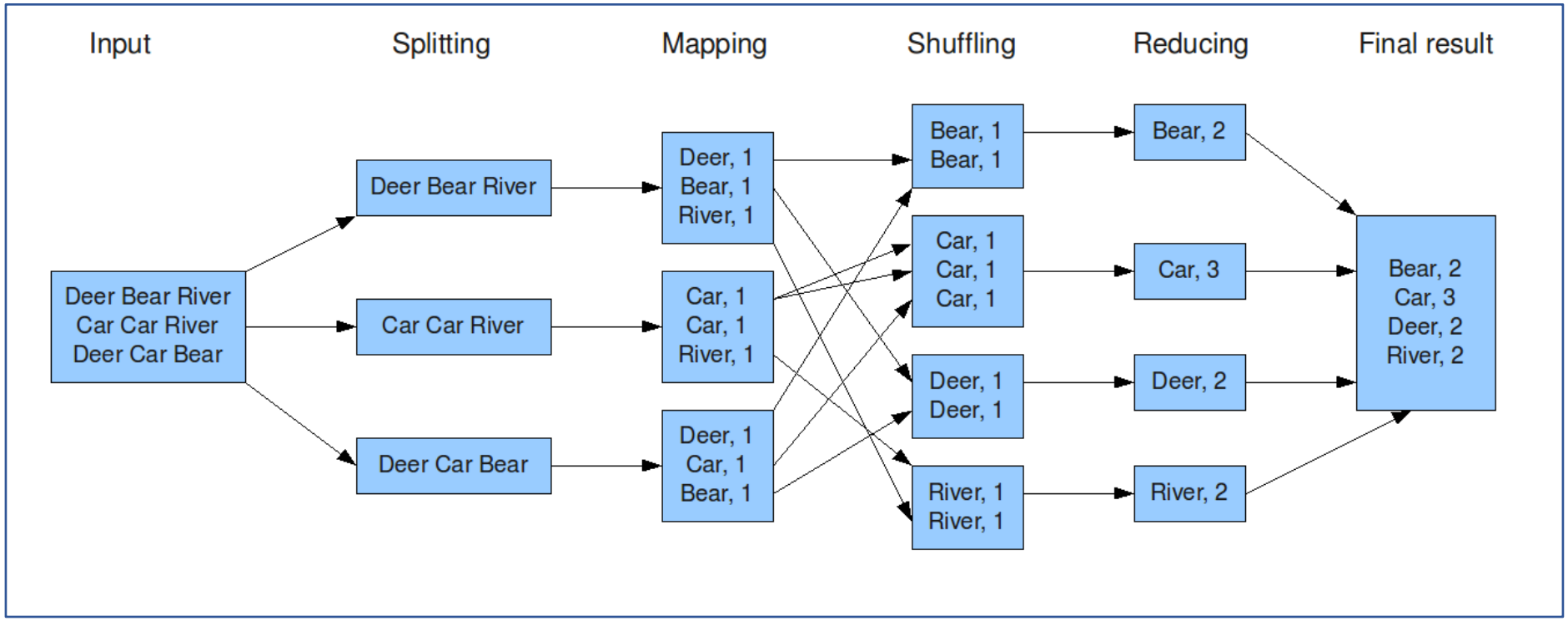
HDFS is a distributed file system designed to store and retrieve large data sets across clusters of computers.



Big Data

❖ MapReduce

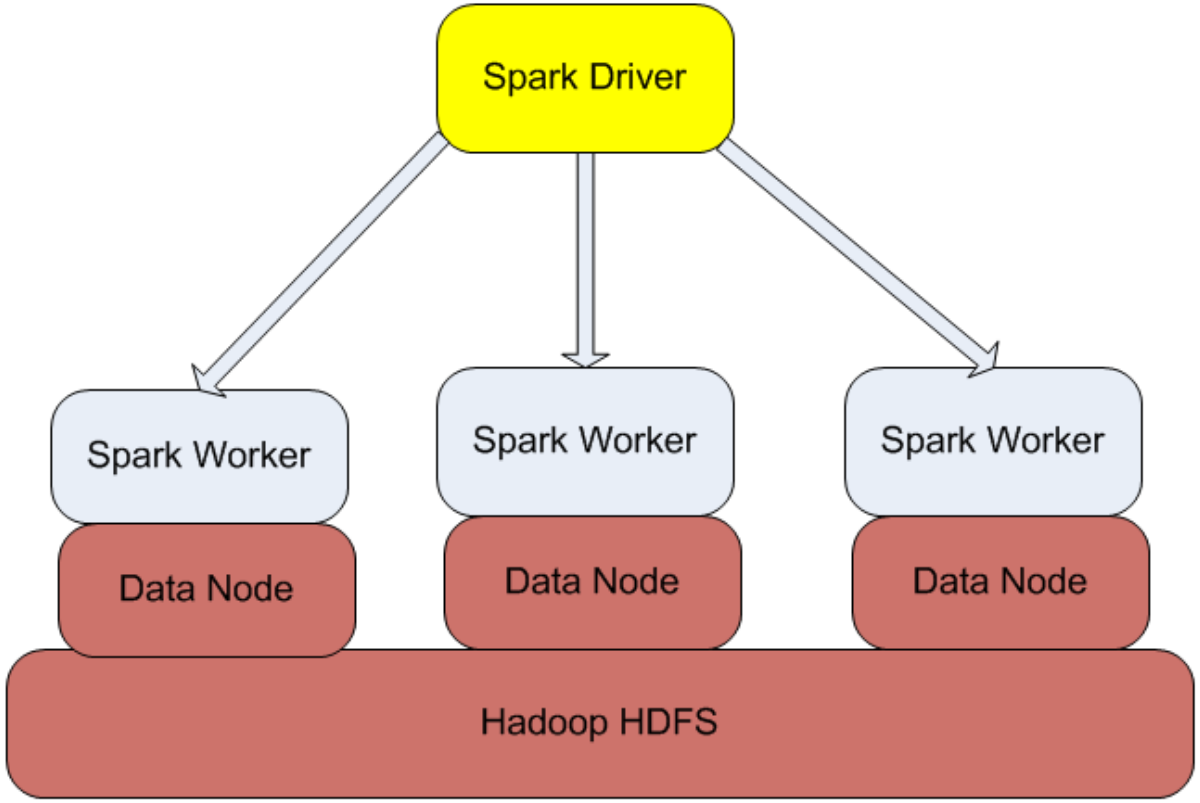
- MapReduce is a programming model designed to process large datasets across clusters of computers.
- This model divides a large data processing job into smaller tasks, which are executed in parallel across multiple machines.
- This significantly speeds up processing and improves system scalability.

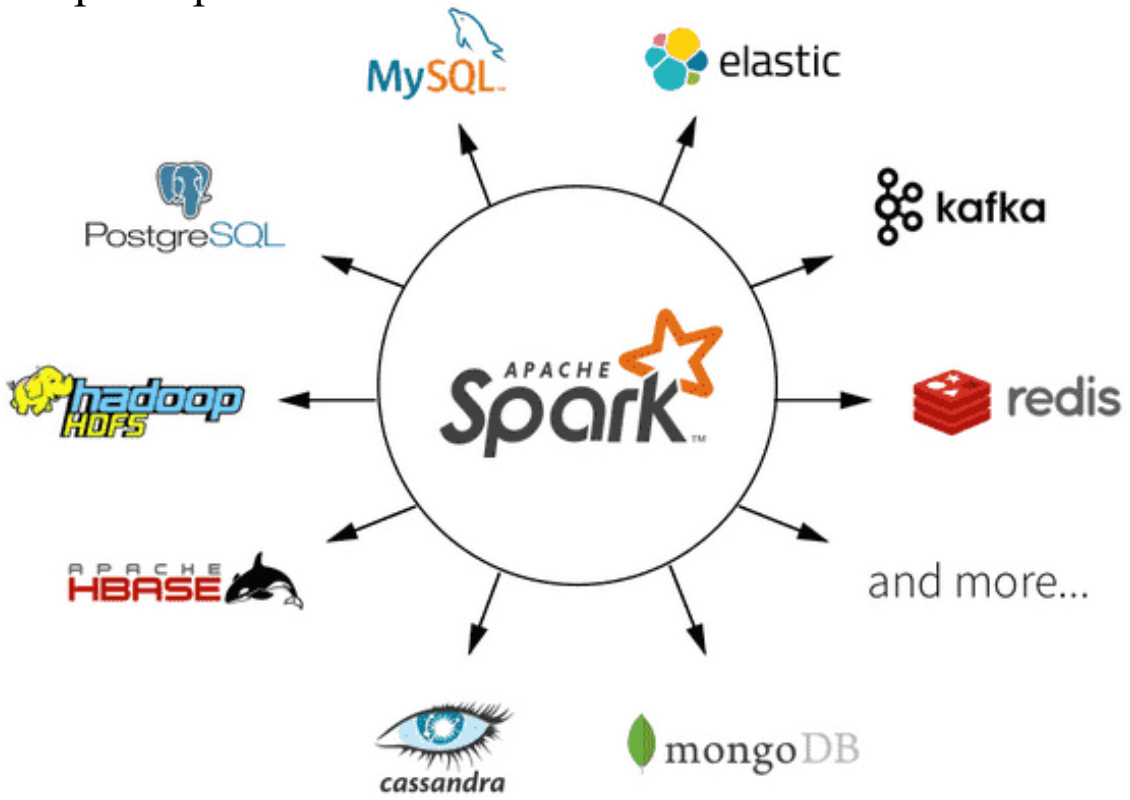


Big Data

❖ Apache Spark

- Spark is a data processing tool that is faster and more flexible than Hadoop.
- It uses in-memory data processing, which significantly speeds up computations.



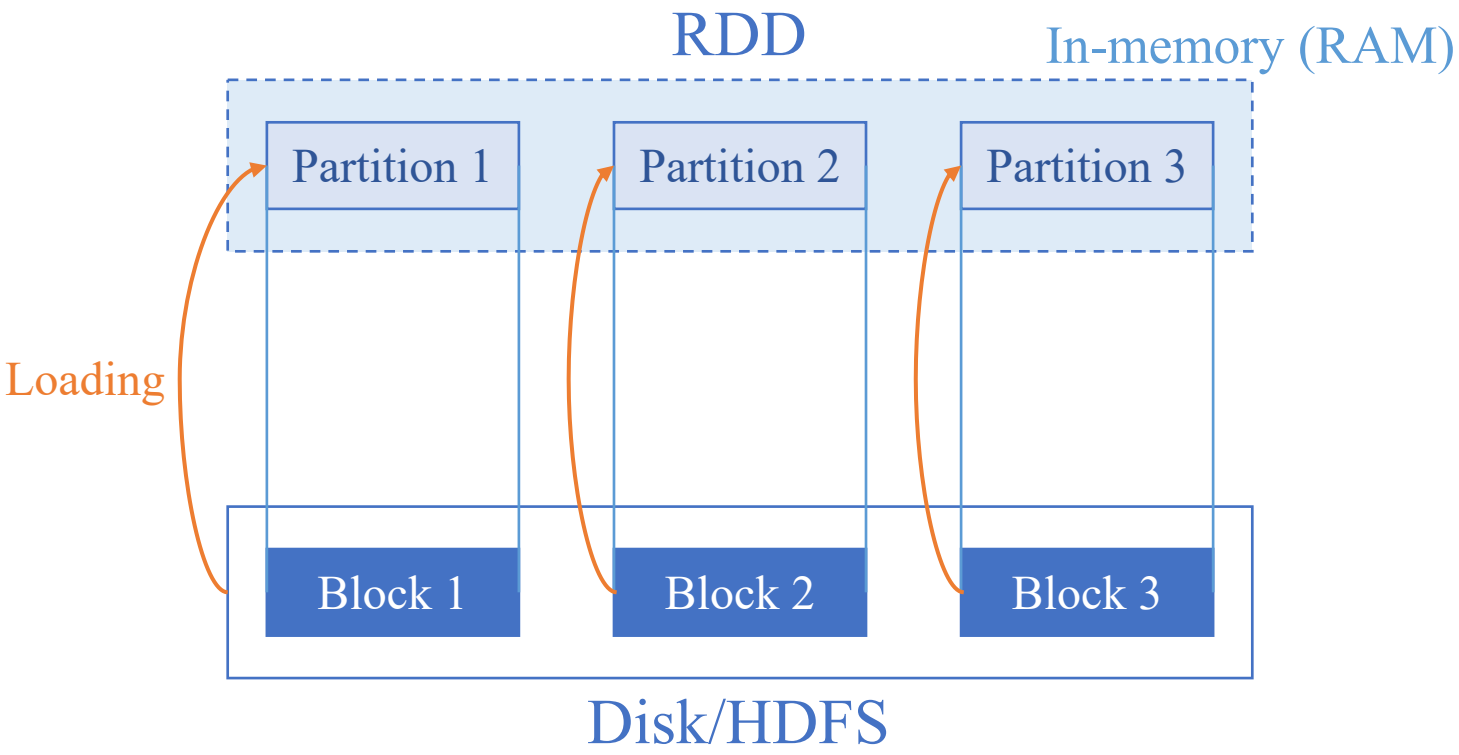


Big Data

❖ Spark RDD (Resilient Distributed Dataset)

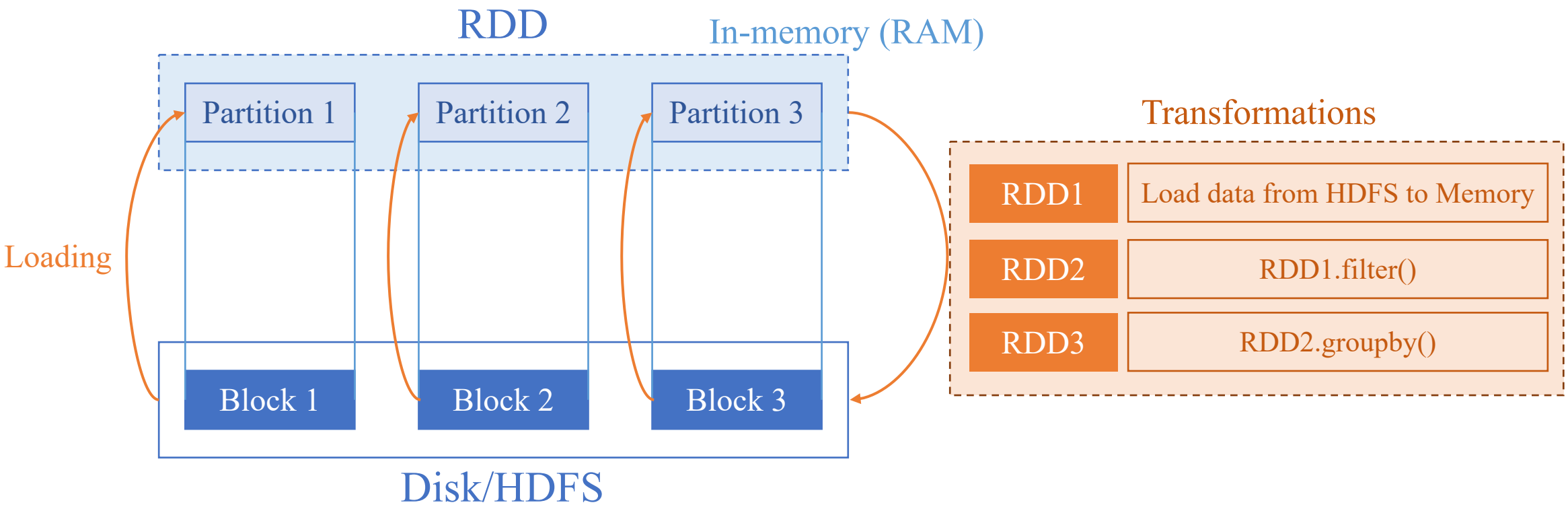
Resilient Distributed Datasets:

- Resilient: Ability to withstand failures
- Distributed: Spanning across multiple machines
- Datasets: Collection of partitioned data eg. Arrays, Tables, Tuples, etc.



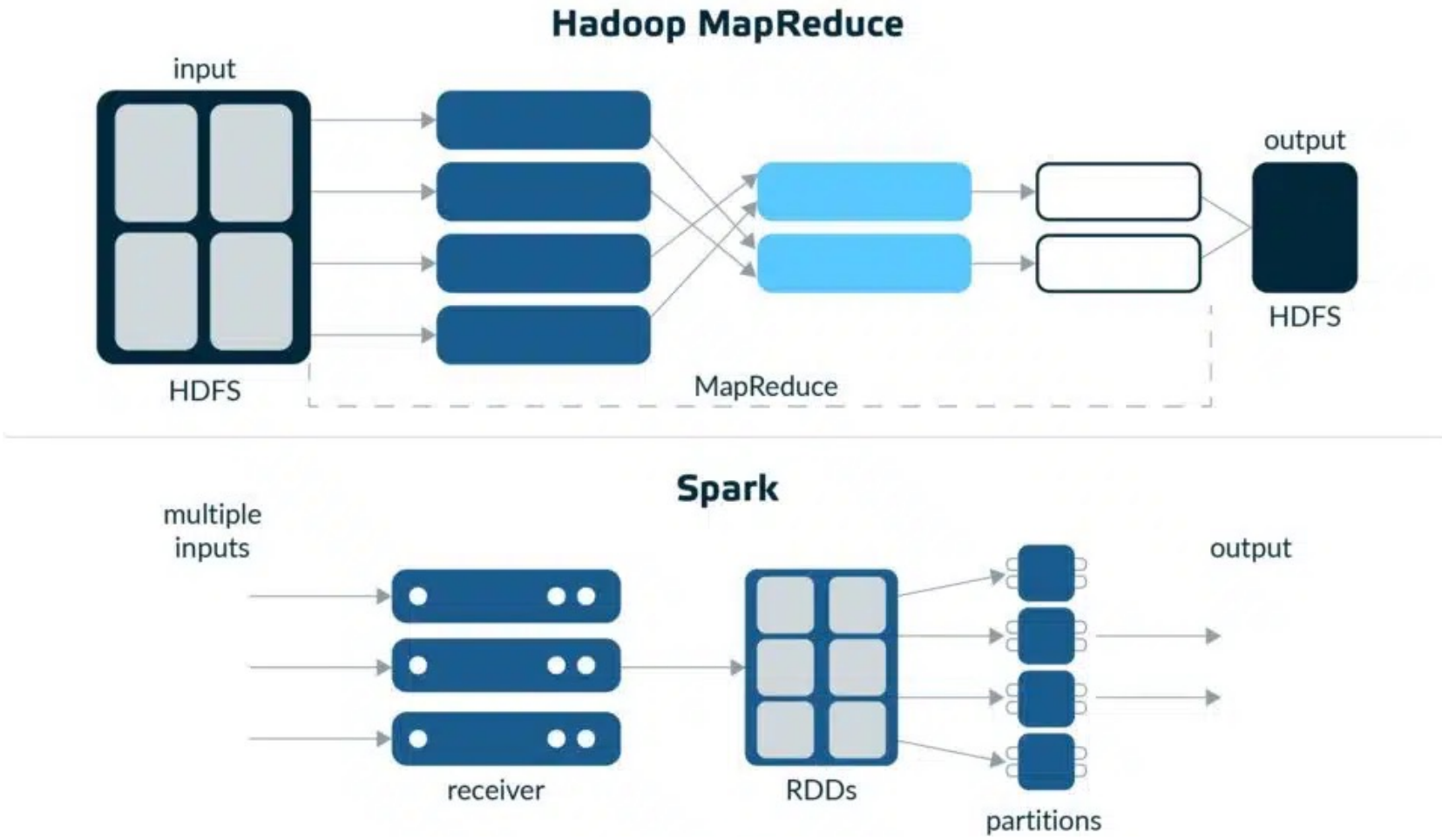
Big Data

❖ Spark RDD (Resilient Distributed Dataset)



Big Data

❖ Spark RDD vs Hadoop MapReduce



Big Data

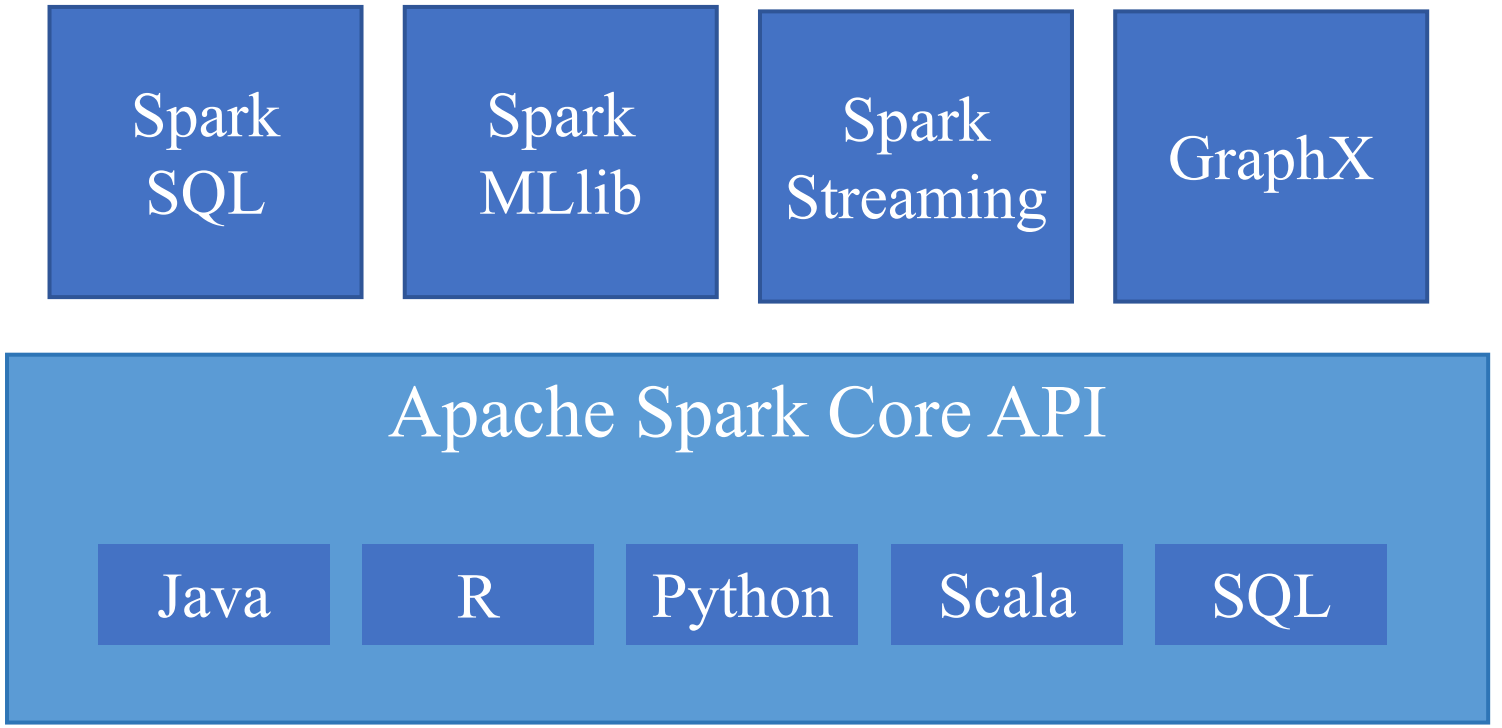
❖ Spark vs Hadoop: Comparison

Parameter	Hadoop	Spark
Intent	Data Processing	Data Analytics
Work Process	Analyses batches of data present in huge volumes	Analyses and processes real-time data
Processing Style	Batch	Real-time
Latency	High	Low
Cost	Less costly due to the MapReduce Model	Is more expensive (in-memory solution)
Ease of Use	Complex to use	Easier to use
Written	Java	Scala

Apache Spark

Apache Spark

❖ Apache Spark Components

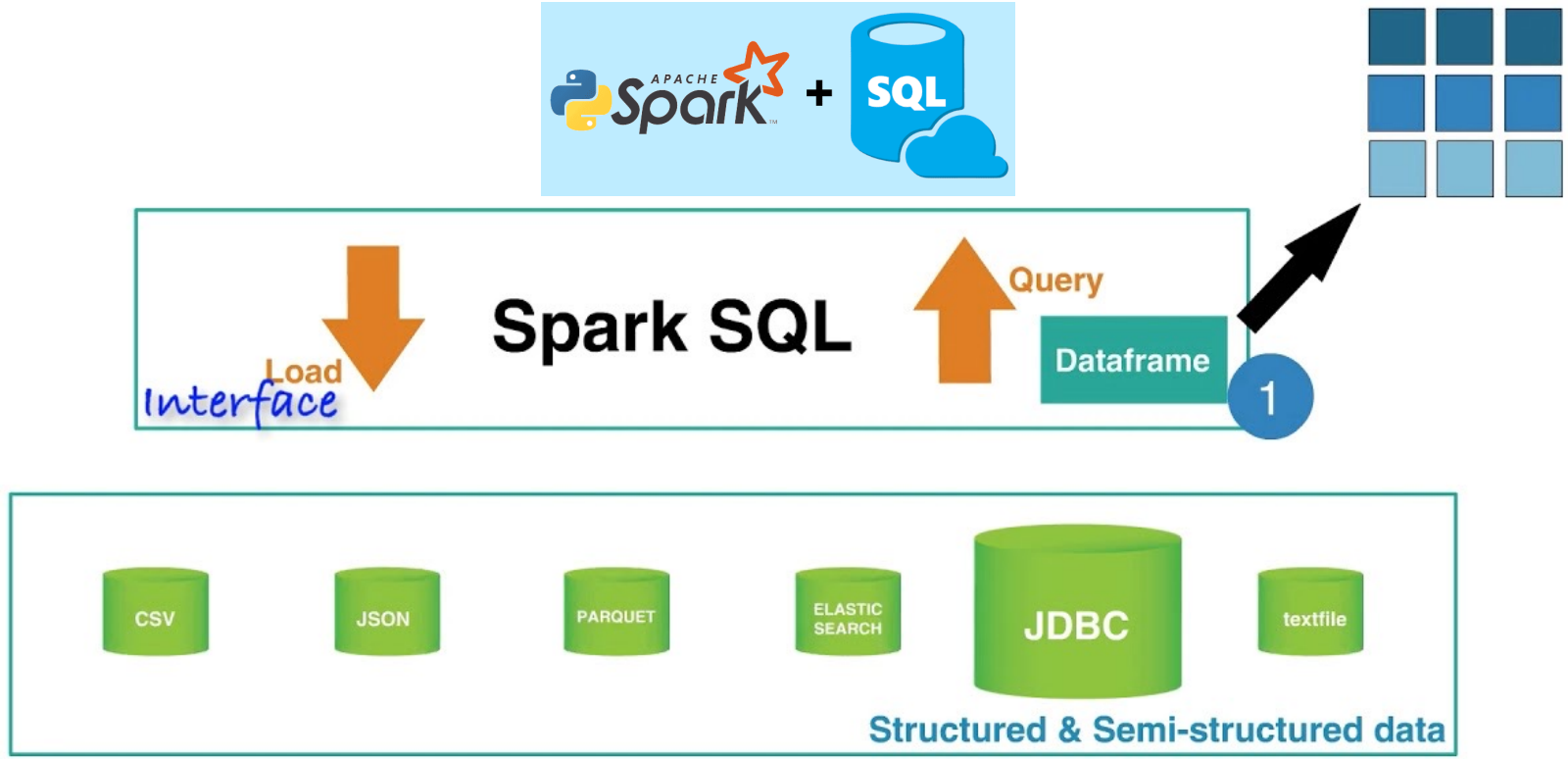


Apache Spark

❖ Spark SQL

Spark SQL integrates SQL queries with Spark's distributed data processing, combining SQL's expressiveness with Spark's speed and scalability.

- It can handle various data formats, including Parquet, JSON, and CSV.
- DataFrames provide a consistent way to work with all these different data sources.

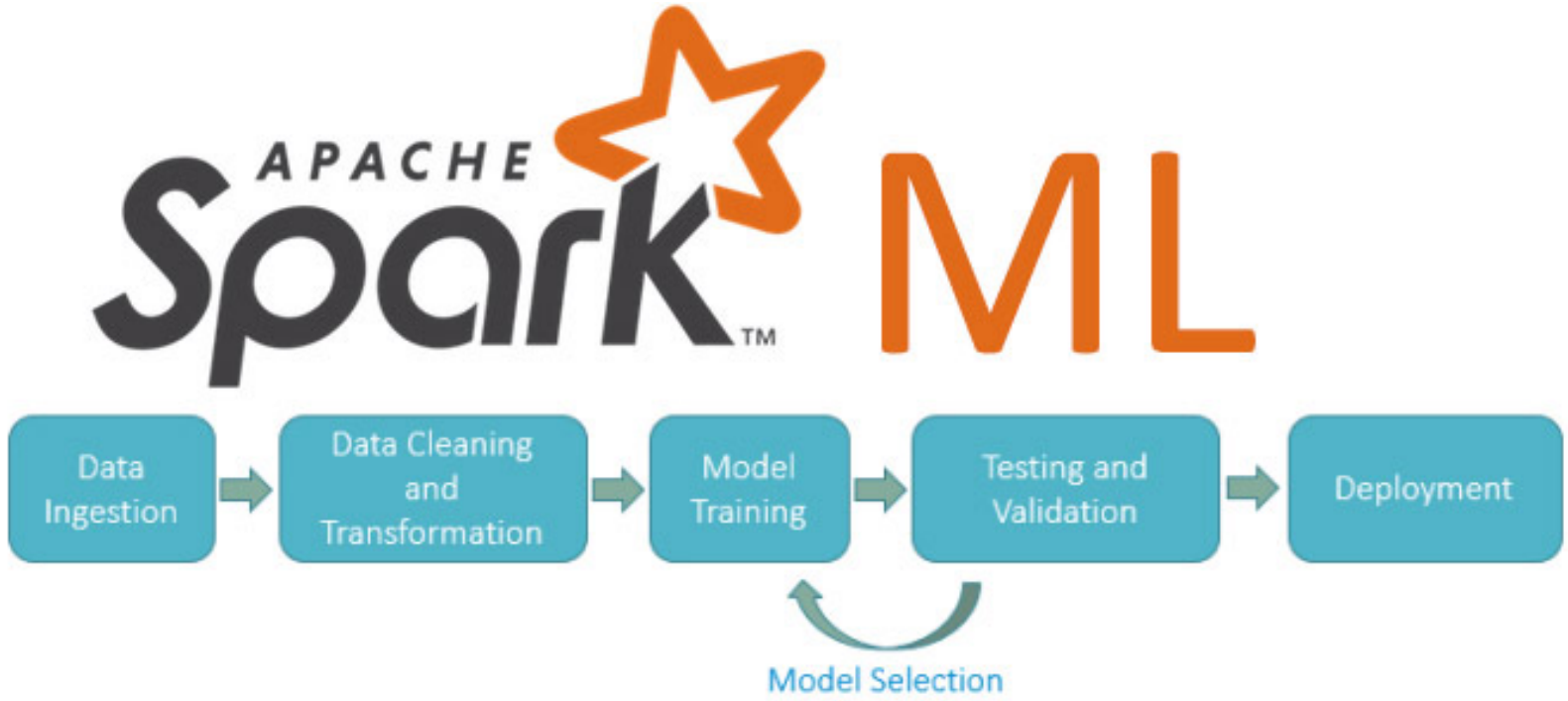


Apache Spark

❖ Spark MLlib (Machine Learning Library)

Various tools provided by MLlib include:

- ML Algorithms: collaborative filtering, classification, and clustering.
- Featurization: feature extraction, transformation, dimensionality reduction, and selection.
- Pipelines: tools for constructing, evaluating, and tuning ML Pipelines.



Apache Spark

❖ Spark Streaming

Spark Streaming is a powerful addition to Spark that allows for efficient processing of live data streams.

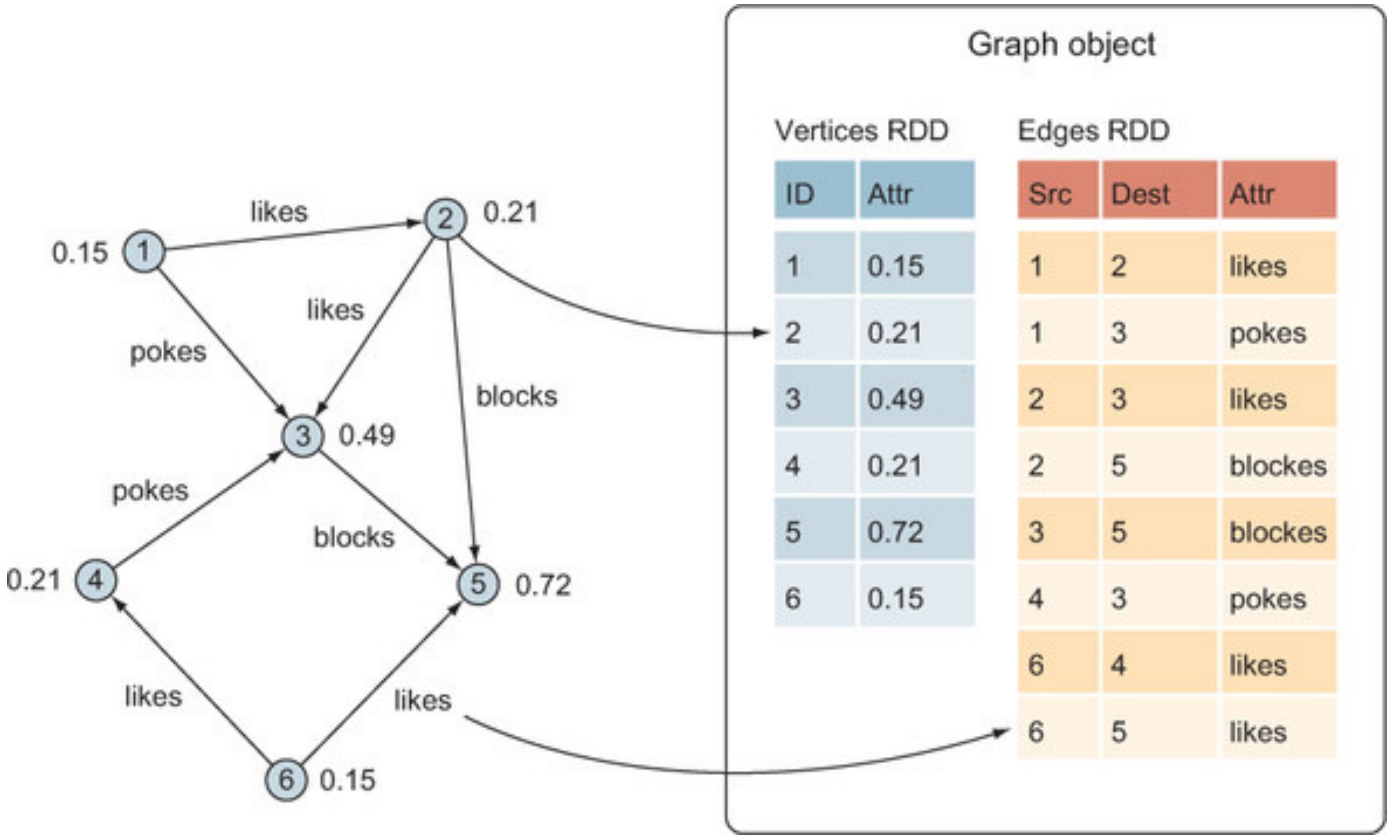
- It can connect to various data sources and perform different types of analysis on the data.
- The processed data can then be stored or visualized in real time.

Apache Spark

❖ Spark GraphX

GraphX in Spark is API for graphs and graph parallel execution.

- It is network graph analytics engine and data store.
- Clustering, classification, traversal, searching, and pathfinding is also possible in graphs.



PySpark

PySpark

❖ Lambda in python

```python
def squared(x: int | float):
    return x**2

a = 5
print(squared(a))
```

Original python function

```python
squared_lambda = lambda x: x**2

a = 5
print(squared_lambda(a))
```

Lambda python function

PySpark

❖ Map, Filter in python

```python
numbers = [1, 2, 3, 4, 5]
```

```python
result = list(map(lambda x: x**2, numbers))
print(result)
```

```python
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

```python
result = []
for num in numbers:
    result.append(squared_lambda(num))
print(result)
```

PySpark

❖ PySpark Layers

Easier with DataFrame

Layer 2 — **High Level APIs**: Spart SQL, DF | Streaming | Mllib | GraphX

Layer 1 — **Spark Core APIs**: Scala | Python | Java | R

Working with RDD level

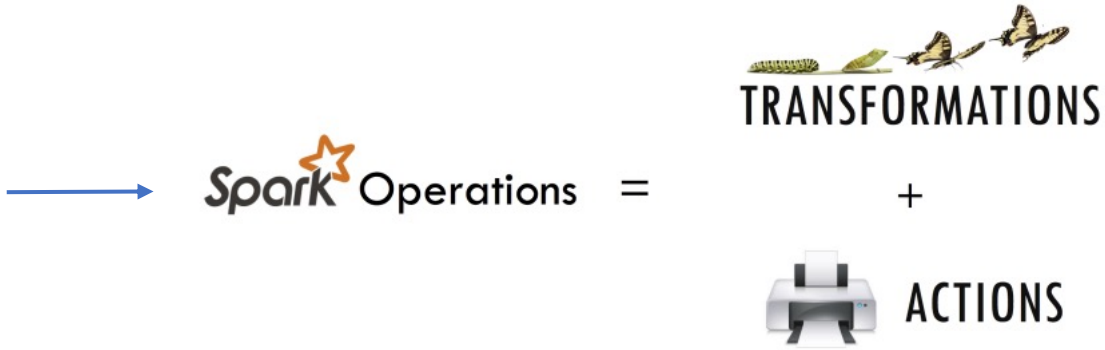
	Data file	Metadata
RDD	RAM	No
DataFrame	RAM	Yes
Spark SQL	Disk	Yes

```
person_id,name,sex,date of birth
100,Penelope Lewis,female,1990-08-31
105,David Simmons,male,1999-12-30
110,Freddie Claridge,male,2002-05-07
115,Robert Mcreynolds,male,1973-12-27
```

PySpark

❖ PySpark RDD

- Transformations create new RDDs
- Actions perform computation on the RDDs



- RDD created by reading data from stable storage.

Basic RDD Transformations:

- Map()
- Filter()
- FlatMap()
- Union()
- Orthers: max(), min(), top()

RDDs

PySpark

❖ RDD: load data

```
num_data = [1,2,3,4,5]

num_rdd = spark.sparkContext.parallelize(num_data)
```

```
text_data = [
    "Hello this is an example of Spark WordCount Example",
    "we are using PySpark",
    "PySpark is a great tool for Big Data Analysis"
]
text_rdd = spark.sparkContext.parallelize(text_data)
```

PySpark

❖ RDD: actions

Basic RDD Actions:

- Collect()
- Take(N)
- First()
- Count()

```
num_data = [1,2,3,4,5]

num_rdd = spark.sparkContext.parallelize(num_data)
```

```
print("RDD count :"+str(num_rdd.count()))
[9] ✓ 0.8s
RDD count :5
```

```
print("RDD take :"+str(num_rdd.take(3)))
[5] ✓ 0.9s
RDD take :[1, 2, 3]
```

```
print("RDD take :"+str(num_rdd.first()))
[7] ✓ 0.5s
RDD take :1
```

PySpark

❖ RDD: map()

```
text_data = [
    "Hello this is an example of Spark WordCount Example",
    "we are using PySpark",
    "PySpark is a great tool for Big Data Analysis"
]

text_rdd = spark.sparkContext.parallelize(text_data)
```

```
text_rdd2 = text_rdd.map(lambda x: (x, len(x.split(" "))))
```

```
text_rdd2.collect()
```

[8] ✓ 0.5s

```
[('Hello this is an example of Spark WordCount Example', 9),
 ('we are using PySpark', 4),
 ('PySpark is a great tool for Big Data Analysis', 9)]
```

PySpark

❖ RDD: flatmap()

```python
text_data = [
    "Hello this is an example of Spark WordCount Example",
    "we are using PySpark",
    "PySpark is a great tool for Big Data Analysis"
]

text_rdd = spark.sparkContext.parallelize(text_data)
```

map

```python
text_rdd2 = text_rdd.map(lambda x: x.split(" "))
text_rdd2.collect()
```

[12] ✓ 0.5s

```
[['Hello',
  'this',
  'is',
  'an',
  'example',
  'of',
  'Spark',
  'WordCount',
  'Example'],
 ['we', 'are', 'using', 'PySpark'],
 ['PySpark', 'is', 'a', 'great', 'tool', 'for', 'Big', 'Data', 'Analysis']]
```

flatMap

```python
text_rdd2_flat = text_rdd.flatMap(lambda x: x.split(" "))
text_rdd2_flat.collect()
```

[13] ✓ 0.4s

```
['Hello',
 'this',
 'is',
 'an',
 'example',
 'of',
 'Spark',
 'WordCount',
 'Example',
 'we',
 'are',
 'using',
 'PySpark',
 'PySpark',
 'is',
 'a',
 'great',
 'tool',
 'for',
 'Big',
 'Data',
 'Analysis']
```

PySpark

❖ RDD: filter()

```
1  num_data = [1,2,3,4,5]
2
3  num_rdd = spark.sparkContext.parallelize(num_data)
```

```
1  num_rdd_filter = num_rdd.filter(lambda x: x % 2 == 0)
```

```
num_rdd_filter.collect()
[8]  ✓ 2.3s
...
...  [2, 4]
```

PySpark

❖ RDD to DataFrame

```python
scores = [("Nguyen Van A", 10.0),
          ("Le Van Quy",9.5),
          ("Tran Duc",8.0),
          ("Pham Thi Nhung",9.),
        ]
rdd = spark.sparkContext.parallelize(scores)
```

```python
scoreColumns = ["name","score"]

df2 = rdd.toDF(scoreColumns)
```

```
df2.printSchema()
df2.show(truncate=False)
```

[7] ✓ 0.7s

```
root
 |-- name: string (nullable = true)
 |-- score: double (nullable = true)

+-------------+-----+
|name         |score|
+-------------+-----+
|Nguyen Van A |10.0 |
|Le Van Quy   |9.5  |
|Tran Duc     |8.0  |
|Pham Thi Nhung|9.0  |
+-------------+-----+
```

Spark SQL

PySpark

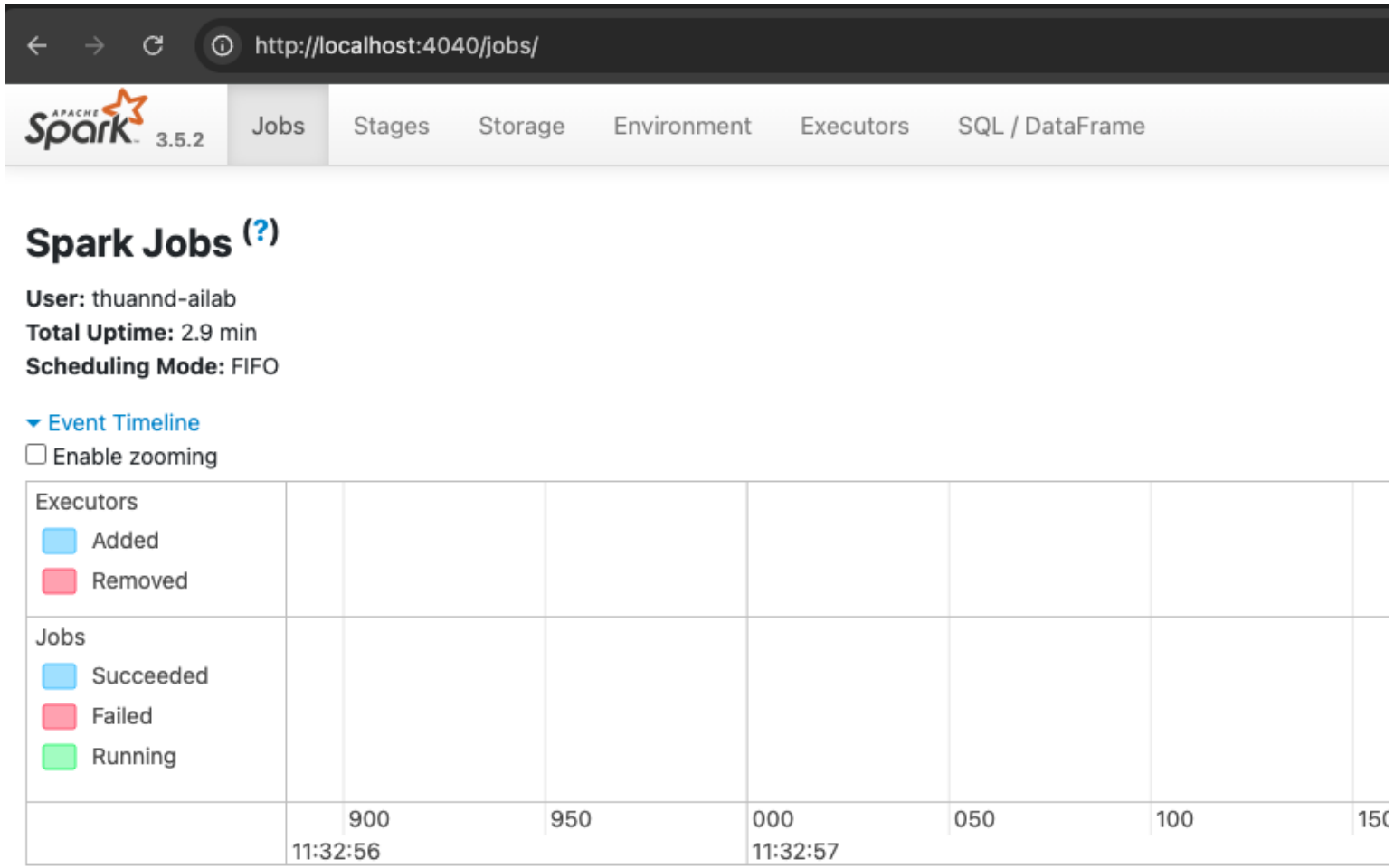
❖ PySpark Start a Session

```python
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('PySparkExample.com').getOrCreate()
```

```
spark
[5] ✓ 0.7s
...
SparkSession - in-memory

SparkContext

Spark UI

Version
      v3.5.2
Master
      local[*]
AppName
      SparkByExamples.com
```

PySpark

❖ PySpark Session UI



PySpark

❖ PySpark Read Data: CSV

```
df_csv = spark.read.csv("./people.csv" ,
                        header=True,
                        inferSchema=True)
```

```
df_csv.printSchema()
```

[5] ✓ 0.0s

```
root
 |-- _c0: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)
```

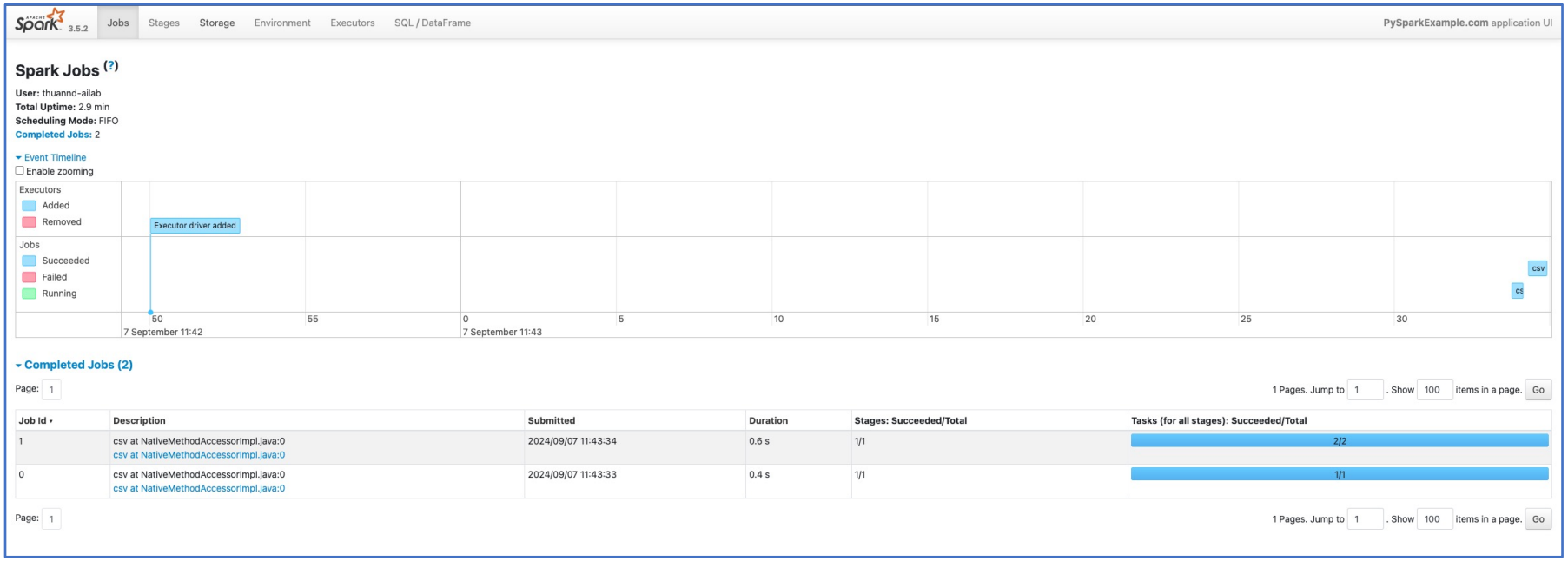
people.csv

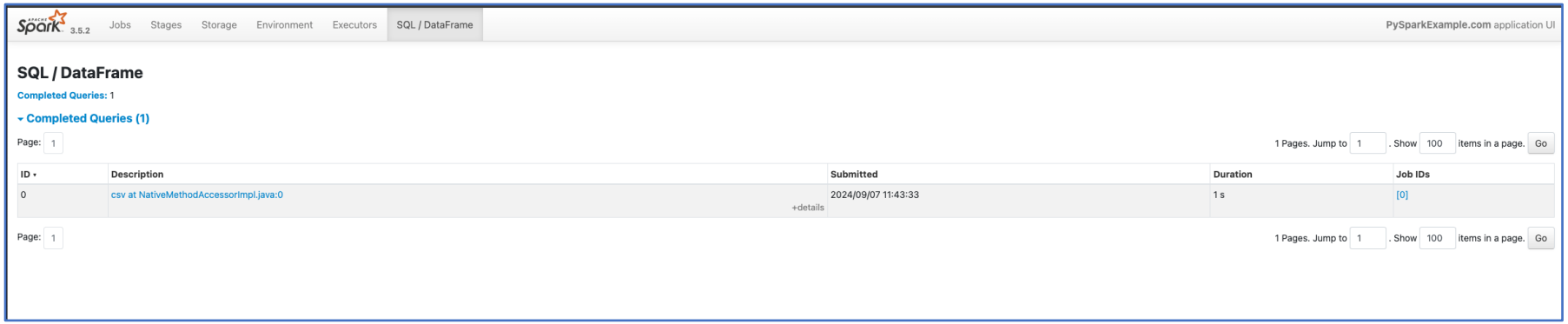
PySpark > people.csv > data

```
,person_id,name,sex,date of birth
0,100,Penelope Lewis,female,1990-08-31
1,101,David Anthony,male,1971-10-14
2,102,Ida Shipp,female,1962-05-24
3,103,Joanna Moore,female,2017-03-10
4,104,Lisandra Ortiz,female,2020-08-05
5,105,David Simmons,male,1999-12-30
6,106,Edward Hudson,male,1983-05-09
7,107,Albert Jones,male,1990-09-13
8,108,Leonard Cavender,male,1958-08-08
9,109,Everett Vadala,male,2005-05-24
10,110,Freddie Claridge,male,2002-05-07
```

PySpark

❖ PySpark Read Data: CSV





PySpark

❖ PySpark Read Data: JSON

```
1  df_json = spark.read.json("./people.json")
```

```
    df_json.printSchema()
[24] ✓ 0.0s

··· root
 |-- date of birth: string (nullable = true)
 |-- name: string (nullable = true)
 |-- person_id: long (nullable = true)
 |-- sex: string (nullable = true)
```

```
1  {"person_id": 100, "name": "Penelope Lewis", "sex": "female", "date of birth": "1990-08-31"}
2  {"person_id": 101, "name": "David Anthony", "sex": "male", "date of birth": "1971-10-14"}
3  {"person_id": 102, "name": "Ida Shipp", "sex": "female", "date of birth": "1962-05-24"}
4  {"person_id": 103, "name": "Joanna Moore", "sex": "female", "date of birth": "2017-03-10"}
5  {"person_id": 104, "name": "Lisandra Ortiz", "sex": "female", "date of birth": "2020-08-05"}
6  {"person_id": 105, "name": "David Simmons", "sex": "male", "date of birth": "1999-12-30"}
7  {"person_id": 106, "name": "Edward Hudson", "sex": "male", "date of birth": "1983-05-09"}
8  {"person_id": 107, "name": "Albert Jones", "sex": "male", "date of birth": "1990-09-13"}
9  {"person_id": 108, "name": "Leonard Cavender", "sex": "male", "date of birth": "1958-08-08"}
10 {"person_id": 109, "name": "Everett Vadala", "sex": "male", "date of birth": "2005-05-24"}
11 {"person_id": 110, "name": "Freddie Claridge", "sex": "male", "date of birth": "2002-05-07"}
```

PySpark

❖ PySpark Read Data: TXT

```
1  df_text = spark.read.text("./text_data.txt")
```

```
df_text.printSchema()
[42]  ✓ 0.0s
root
 |-- value: string (nullable = true)
```

```
df_text.show()
[54]  ✓ 0.1s
+--------------------+
|               value|
+--------------------+
| Dữ liệu có cấu trúc|
|Dữ liệu có cấu tr...|
|để quản lý và tìm...|
|lưu trữ và xử lý ...|
|Các thành phần củ...|
|cho phép các nhà ...|
|đơn giản để tìm k...|
|Dữ liệu phi cấu trúc|
|Dữ liệu phi cấu t...|
```

```
1  Dữ liệu có cấu trúc
2  Dữ liệu có cấu trúc được xem là dữ liệu đơn giản nhất
3  để quản lý và tìm kiếm. Nó là những dữ liệu có thể truy cập,
4  lưu trữ và xử lý ở định dạng cố định.
5  Các thành phần của dữ liệu có cấu trúc được phân loại dễ dàng,
6  cho phép các nhà thiết kế và quản trị viên cơ sở dữ liệu xác định các thuật toán
7  đơn giản để tìm kiếm và phân tích.
8  Dữ liệu phi cấu trúc
9  Dữ liệu phi cấu trúc là bất kỳ tập hợp dữ liệu nào không được tổ chức
10 hoặc xác định rõ ràng. Loại dữ liệu này hỗn loạn, khó xử lý, khó hiểu và đánh giá.
11 Nó không có cấu trúc cố định và có thể thay đổi vào những thời điểm khác nhau.
12 Dữ liệu phi cấu trúc bao gồm các nhận xét,
13 tweet, lượt chia sẻ, bài đăng trên mạng xã hội,
```

PySpark

❖ PySpark DataFrame: select()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```

[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|  Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|   David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|       Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|    Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|  Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|   David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|   Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|    Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108|Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|  Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110|Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|   Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|      Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119|Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_csv.select("name").show(truncate=True)
```

[4] ✓ 0.1s

```
+----------------+
|            name|
+----------------+
|  Penelope Lewis|
|   David Anthony|
|       Ida Shipp|
|    Joanna Moore|
|  Lisandra Ortiz|
|   David Simmons|
|   Edward Hudson|
|    Albert Jones|
|Leonard Cavender|
|  Everett Vadala|
|Freddie Claridge|
|Annabelle Rosseau|
|   Eulah Emanuel|
|      Shaun Love|
|Alejandro Brennan|
|Robert Mcreynolds|
|  Carla Spickard|
|Florence Eberhart|
|    Tina Gaskins|
|Florence Mulhern|
+----------------+
only showing top 20 rows
```

PySpark

❖ PySpark DataFrame: filter()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```

[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|  Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|   David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|       Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|    Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|  Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|   David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|   Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|    Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108|Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|  Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110|Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|   Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|      Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119|Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_csv.filter(df_csv.sex == "male").show(truncate=True)
```

[6] ✓ 0.2s

```
+-----+---------+----------------+----+-------------------+
|index|person_id|            name| sex|      date of birth|
+-----+---------+----------------+----+-------------------+
|    1|      101|   David Anthony|male|1971-10-14 00:00:00|
|    5|      105|   David Simmons|male|1999-12-30 00:00:00|
|    6|      106|   Edward Hudson|male|1983-05-09 00:00:00|
|    7|      107|    Albert Jones|male|1990-09-13 00:00:00|
|    8|      108|Leonard Cavender|male|1958-08-08 00:00:00|
|    9|      109|  Everett Vadala|male|2005-05-24 00:00:00|
|   10|      110|Freddie Claridge|male|2002-05-07 00:00:00|
|   13|      113|      Shaun Love|male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|male|1980-12-22 00:00:00|
|   15|      115|Robert Mcreynolds|male|1973-12-27 00:00:00|
|   20|      120|    Joel Smith|male|1996-08-13 00:00:00|
|   23|      123|   Angel Moher|male|2017-12-22 00:00:00|
|   24|      124|Charles Leonard|male|1972-03-09 00:00:00|
|   25|      125|    Mark Miller|male|1976-05-11 00:00:00|
|   29|      129|   David Bishop|male|1960-10-18 00:00:00|
|   31|      131|   Joseph Windus|male|2029-05-27 00:00:00|
|   32|      132|Christopher Gilbert|male|2010-09-21 00:00:00|
|   33|      133|Robert Salisbury|male|1965-06-22 00:00:00|
|   36|      136|   Wilbert Glass|male|1976-10-10 00:00:00|
|   40|      140|      Juan Dunn|male|1969-10-06 00:00:00|
+-----+---------+----------------+----+-------------------+
only showing top 20 rows
```

PySpark

❖ PySpark DataFrame: groupby()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```
[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|   Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|    David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|        Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|     Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|   Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|    David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|    Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|     Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108| Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|   Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110| Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|    Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|       Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119| Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_groupby_sex = df_csv.groupby('sex')
print(type(df_groupby_sex))
```
[7] ✓ 0.0s

```
<class 'pyspark.sql.group.GroupedData'>
```

```
df_groupby_sex.count().show()
```
[8] ✓ 0.9s

```
+------+-----+
|sex   |count|
+------+-----+
|NULL  |1920 |
|female|49014|
|male  |49066|
+------+-----+
```

PySpark

❖ PySpark SQL: SQL query

```
1  df_csv.createOrReplaceTempView("people")
```

```
1  query = '''SELECT name, sex
2  FROM people'''
3
4  df2 = spark.sql(query)
```

```
[8]  df2.show(truncate=True)

+----------------+------+
|            name|   sex|
+----------------+------+
|   Penelope Lewis|female|
|    David Anthony|  male|
|        Ida Shipp|female|
|     Joanna Moore|female|
|   Lisandra Ortiz|female|
|    David Simmons|  male|
|    Edward Hudson|  male|
|     Albert Jones|  male|
| Leonard Cavender|  male|
|   Everett Vadala|  male|
| Freddie Claridge|  male|
|Annabelle Rosseau|female|
|    Eulah Emanuel|female|
|       Shaun Love|  male|
|Alejandro Brennan|  male|
|Robert Mcreynolds|  male|
|   Carla Spickard|female|
|Florence Eberhart|female|
|     Tina Gaskins|female|
| Florence Mulhern|female|
+----------------+------+
only showing top 20 rows
```

PySpark

❖ PySpark SQL: grouping

```
1  df_csv.createOrReplaceTempView("people")
```

```
1  query = '''SELECT sex, COUNT(person_id) FROM people GROUP BY sex'''
2
3  df2 = spark.sql(query)
```

```
df2.show()
[23]  ✓ 0.3s
...  +------+----------------+
     |   sex|count(person_id)|
     +------+----------------+
     |  NULL|            1920|
     |female|           49014|
     |  male|           49066|
     +------+----------------+
```

PySpark

❖ PySpark SQL: filter

```
1  df_csv.createOrReplaceTempView("people")
```

```
1  query = '''SELECT name, sex
2  FROM people
3  WHERE sex = "female"'''
4
5  df2 = spark.sql(query)
```

```
df2.show(truncate=True)
[18]  ✓ 0.2s
...  +----------------+------+
     |            name|   sex|
     +----------------+------+
     |   Penelope Lewis|female|
     |        Ida Shipp|female|
     |     Joanna Moore|female|
     |   Lisandra Ortiz|female|
     |Annabelle Rosseau|female|
     |    Eulah Emanuel|female|
     |   Carla Spickard|female|
     |Florence Eberhart|female|
     |     Tina Gaskins|female|
     | Florence Mulhern|female|
     |    Evelyn Kriner|female|
     |     Heather Luce|female|
     |      Marion Baca|female|
     |       Devona Kay|female|
     |   Betty Endicott|female|
     |        Jane Ross|female|
     |   Pauline Steele|female|
     |     Anne Novotny|female|
     |      Carol Noble|female|
     | Constance Fulmer|female|
     +----------------+------+
     only showing top 20 rows
```

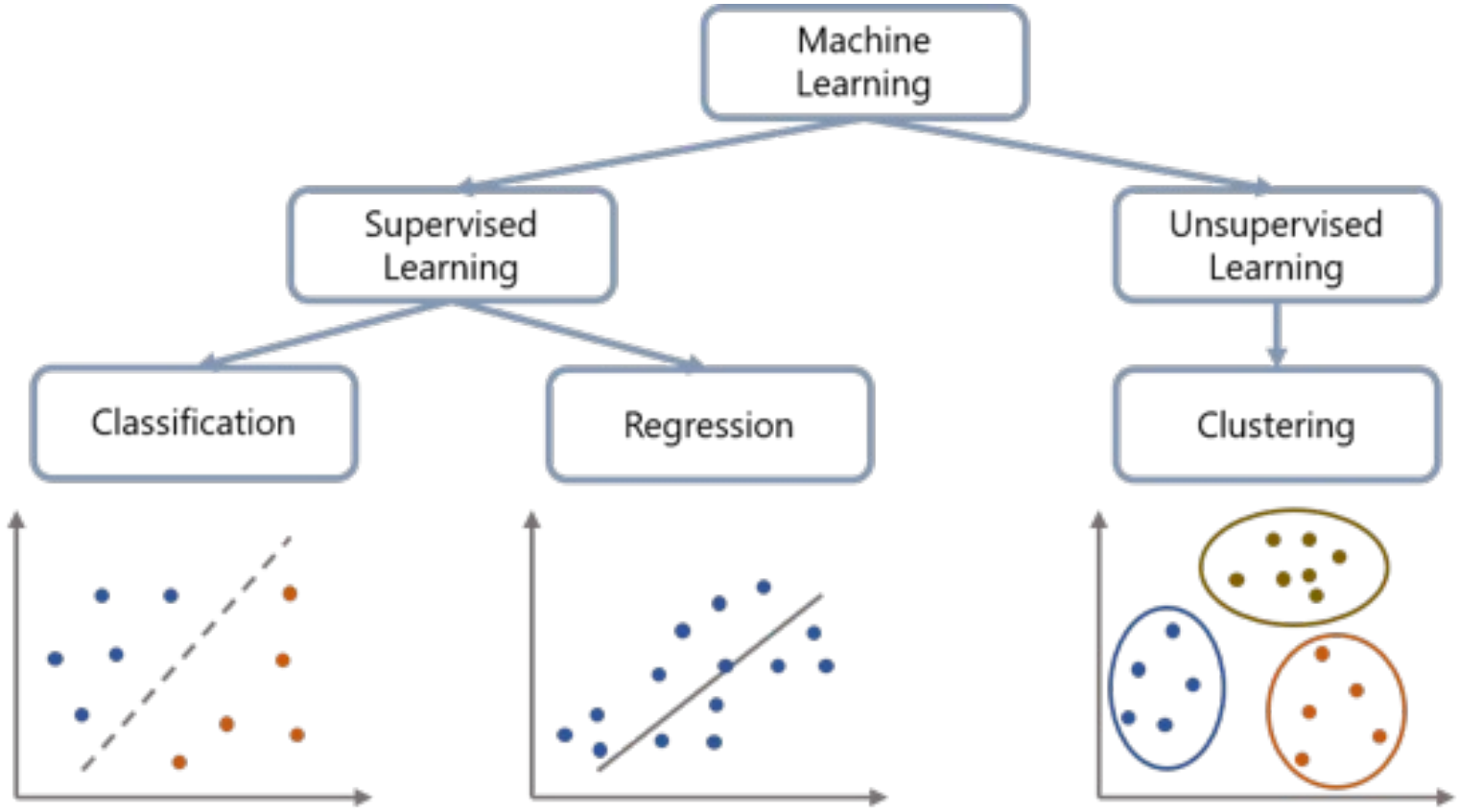
AI VIETNAM
All-in-One Course
(TA Session)

Spark MLlib

PySpark

❖ PySpark MLlib

PySpark MLlib is the machine learning library in PySpark that enables scalable machine learning on large datasets.



PySpark

❖ Why PySpark MLlib?

PySpark MLlib offers big data processing, and many machine learning tools, and works well with other Spark features, making it a powerful choice for large-scale data analysis and modeling.

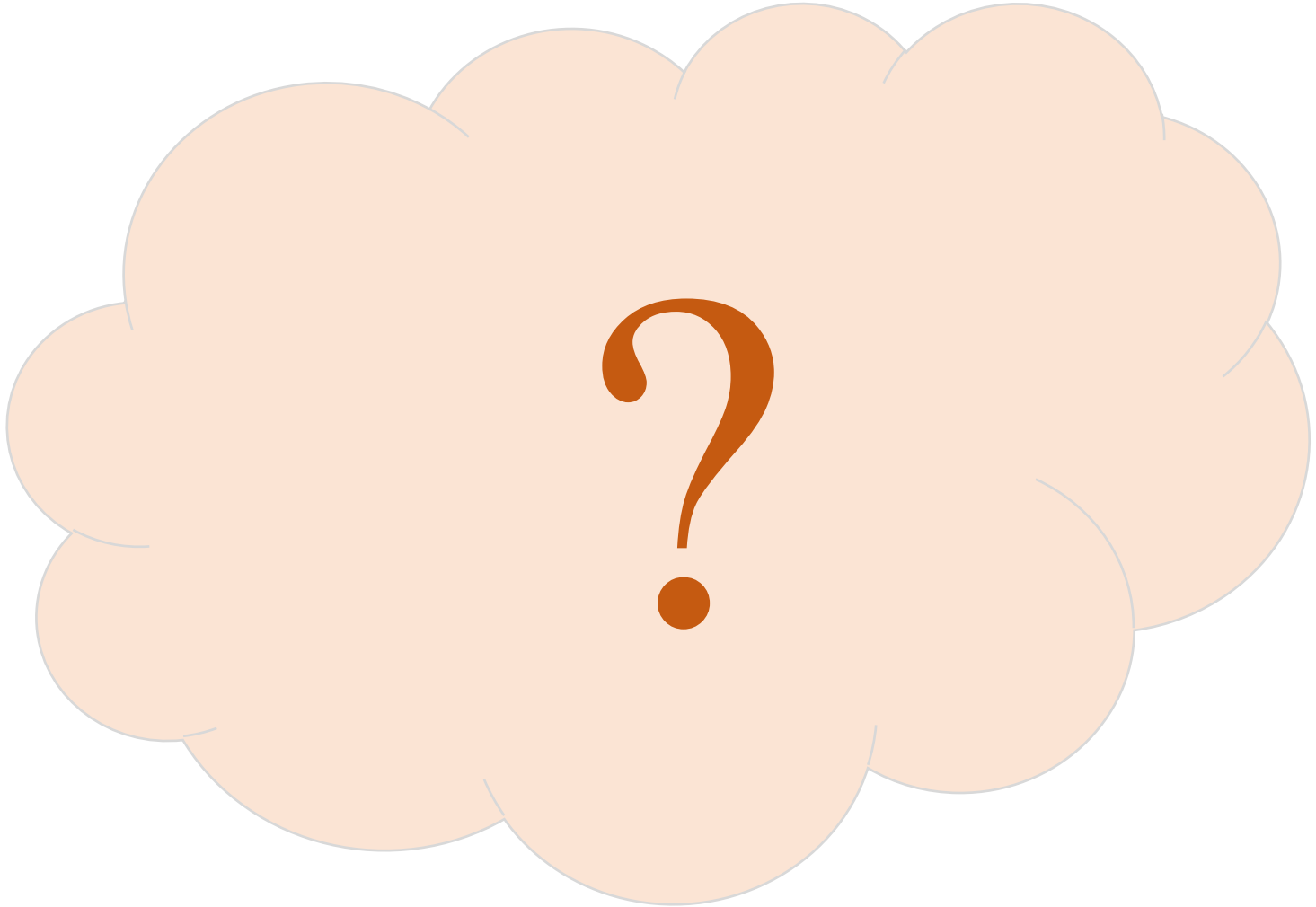
Hands-on

Hands-on

❖ Coding

https://colab.research.google.com/drive/1GOFV4NslkMkKAfOzgmcfD-BpUVkvXfbu?usp=sharing

Question




Thank you!